

Kandidatrapport

Fault-Tolerant HFT Architecture via Process Isolation

Oliver Drevet
Jonathan Eriksson
Axel Genar
Mhd Khaled Helwany
Noah Ljungberg
Oskar Näs
Alanas Pakulnis
Erik Törnudd
Lucas Wihren

Handledare : Christoffer Holm
Examinator : Lena Buffoni

Upphovsrätt

Detta dokument hålls tillgängligt på Internet - eller dess framtida ersättare - under 25 år från publiceringsdatum under förutsättning att inga extraordinära omständigheter uppstår.

Tillgång till dokumentet innebär tillstånd för var och en att läsa, ladda ner, skriva ut enstaka kopior för enskilt bruk och att använda det oförändrat för ickekommersiell forskning och för undervisning. Överföring av upphovsrätten vid en senare tidpunkt kan inte upphäva detta tillstånd. All annan användning av dokumentet kräver upphovsmannens medgivande. För att garantera äktheten, säkerheten och tillgängligheten finns lösningar av teknisk och administrativ art.

Upphovsmannens ideella rätt innefattar rätt att bli nämnd som upphovsman i den omfattning som god sed kräver vid användning av dokumentet på ovan beskrivna sätt samt skydd mot att dokumentet ändras eller presenteras i sådan form eller i sådant sammanhang som är kränkande för upphovsmannens litterära eller konstnärliga anseende eller egenart.

För ytterligare information om Linköping University Electronic Press se förlagets hemsida <http://www.ep.liu.se/>.

Copyright

The publishers will keep this document online on the Internet - or its possible replacement - for a period of 25 years starting from the date of publication barring exceptional circumstances.

The online availability of the document implies permanent permission for anyone to read, to download, or to print out single copies for his/hers own use and to use it unchanged for non-commercial research and educational purpose. Subsequent transfers of copyright cannot revoke this permission. All other uses of the document are conditional upon the consent of the copyright owner. The publisher has taken technical and administrative measures to assure authenticity, security and accessibility.

According to intellectual property law the author has the right to be mentioned when his/her work is accessed as described above and to be protected against infringement.

For additional information about the Linköping University Electronic Press and its procedures for publication and for assurance of document integrity, please refer to its www home page: <http://www.ep.liu.se/>.

© Oliver Drevet
Jonathan Eriksson
Axel Genar
Mhd Khaled Helwany

Noah Ljungberg
Oskar Näs
Alanas Pakulnis
Erik Törnudd
Lucas Wíhren

Sammanfattning

Rapporten beskriver kandidatprojektet *Fault-Tolerant HFT Architecture via Process Isolation*. Projektet utfördes på uppdrag från Alexandria Invest AB och innebar en övergripande omstrukturering av en högfrekvenshandelsplattform som använde en monolitisk enkeltrådad struktur till att bli en felisolerad multiprocessarkitektur. Syftet med projektet är att skapa ett mer robust system som motsvarar det befintliga i exekveringshastighet samt att samla kunskap och erfarenheter till framtida projekt. Omstruktureringen gjordes för att öka systemets robusthet. Under projektets gång användes C++23 i den etablerade kodbasen för att uppnå de strikta kraven kring latens som krävs för att vara konkurrenskraftig inom högfrekvenshandel. Under utvecklingens gång genom samråd med kunden togs beslutet att nyttja stora språkmodeller som *Codex* och *Gemini* för att snabbare komma igång med arbetet och för att producera ett resultat av tillräcklig kvalitet. De resultat som nåddes var att IPC-lagret introducerar ett latenspåslag jämfört med in-process-körning, med en uppmätt median på 36 μ s för *out-of-process* mot 0,69 μ s in-process, men arkitekturen håller sig inom de latenskrav som definierats för feltolerant strategikörning. Återhämtningstiden efter att en process kraschat uppmättes till 118 ms vilket passerar målet på 1 s med god marginal. Rapporten lyfter de metoder, hjälpmedel, arbetsprocesser och nödvändig teori som använts under projektets gång. Ytterligare förs en djupgående diskussion kring de fördelar och nackdelar som tillkommer i brukandet av stora språkmodeller och de konsekvenser för lärande, arbetsprocess och kvalitet dessa brukande får.

Abstract

This report presents the bachelor thesis project *Fault-Tolerant HFT Architecture via Process Isolation*, conducted on behalf of Alexandria Invest AB. The project involved a comprehensive restructuring of a high-frequency trading platform, transitioning from a monolithic single-threaded architecture to a fault-isolated multi-process architecture, with the goal of increasing system robustness. *Modern C++* was used throughout the established codebase to meet the strict latency requirements necessary to remain competitive in high-frequency trading. During development, the decision was made in consultation with the client to employ large language models such as *Codex* and *Gemini* to accelerate the initial stages of development without sacrificing result quality.

The goal of the project was to create a more robust system that matches the existing one in execution speed, while also gathering knowledge and experience for future projects. The results show that the IPC layer introduces a latency overhead compared to in-process execution, with a measured median of 36 μ s for out-of-process versus 0.69 μ s in-process, but the architecture remains within the latency requirements defined for fault-tolerant strategy execution. Recovery time following a process crash was measured at 118 ms, passing the target of 1 s with considerable margin.

The report describes the methods, tools, processes and theory applied throughout the project, and includes an in-depth discussion of the advantages and disadvantages of using large language models, and the consequences their use has on learning, workflow and quality.

Författarens tack

Vi vill tacka vår handledare Christoffer Holm och examinator Lena Buffoni för deras goda stöd och rådgivning genom hela projektet. Ett särskilt tack till vår kund Alexandria Invest AB för ett väl genomfört samarbete.

Innehåll

Sammanfattning	iv
Författarens tack	v
Innehåll	vi
Figurer	ix
Tabeller	x
1 Inledning	4
1.1 Motivering	4
1.2 Syfte	4
1.3 Frågeställningar	5
1.4 Avgränsningar	5
1.5 Kontext	5
1.6 Dokumentation	6
2 Bakgrund	7
2.1 Kundens bakgrund	7
2.2 Uppdrag	7
2.3 Gruppens tidigare erfarenheter	8
3 Teori	9
3.1 Utvecklingsverktyg	9
3.1.1 C++23	9
3.1.2 Git	9
3.1.3 GitLab	9
3.1.4 Docker	10
3.2 Arbetsmetoder	10
3.2.1 Kanbanbräde	10
3.2.2 Scrum	10
3.2.3 Agentiskt ingenjörskap	11
3.3 Trådar, processer och I/O	11
3.3.1 Atomiska operationer	11
3.3.2 SPSC-Ring struktur	11
3.3.3 Låsfritt system	12
3.3.4 Skillnad i Processer och trådar	12
3.3.5 Epoll	12
3.4 High Frequency Trading (HFT)	12
3.5 Kommunikation mellan processer (IPC)	13
4 Metod	14

4.1	Värde för kunden	14
4.2	Metodval	14
4.2.1	Arbetsmetodiken som användes	14
4.2.2	Planeringsmöte	15
4.2.3	Utvärderingsmöte	15
4.2.4	Issues som kanban	15
4.2.5	Överlämning av kodbas	15
4.2.6	Användning av agentiskt ingenjörskap	15
4.2.7	Insamling av erfarenheter	17
4.2.8	Systemanatomi	17
4.3	Testning	18
4.3.1	Funktionalitet	18
4.3.2	Hur tester användes för prestandamätningar	19
4.3.3	Integration	20
4.3.4	Riktlinjer för modul- och enhetstestning	20
4.4	Roller	20
4.5	Utbildning	21
4.5.1	Teoretisk genomgång	21
4.5.2	Praktiska övningar	22
4.5.3	Hur modern C++ ska användas	22
5	Resultat	23
5.1	Hur en processisolerad arkitektur skapar värde för kunden	23
5.1.1	Mätresultat	25
5.2	Processbeskrivning	26
5.2.1	Scrum och Kanban i praktiken	26
5.2.2	Huvudsakliga issues	26
5.2.3	Utvärdering av processer	27
5.2.4	Huvudprocess	27
5.2.5	Förändring av huvudprocess	28
5.3	Erfarenheter från projektets genomförande	28
6	Diskussion	30
6.1	Hur skapar arkitekturen värde för kunden	30
6.2	Erfarenheter givna av projektet	30
6.2.1	Utbildning och det tidiga skedet	31
6.3	Systemanatomi som stöd	31
6.4	Modern C++ roll i projektet	31
6.5	Agentiskt ingenjörskap	32
6.5.1	Varför en mänsklig grupp behövdes	32
6.5.2	Långsiktig hållbarhet av AI-assisterad utveckling	33
6.5.3	Begränsningar i testningens täckning	33
6.6	Metod	33
6.7	Arbetet i ett vidare sammanhang	34
7	Slutsats	36
7.1	Besvarande av frågeställningar	36
7.1.1	Värde för kunden	36
7.1.2	Erfarenheter	36
7.1.3	Stöd av systemanatomin	37
7.1.4	Användning av modern C++ inom multitrådning och prestandaorienterad kod	37
7.2	Utbildning för kandidatprojekt	37

7.3	Konsekvenser för målgruppen	38
7.4	Framtida arbete	38
Referenser		39

Figurer

2.1	Översikt över arkitektur.	8
3.1	SPSC-ringbuffert (<i>Single-Producer Single-Consumer</i>)	12
4.1	Linjärt beroendeflöde mellan komponenter	18
5.1	Jämförelse mellan in-process- och out-of-process-exekvering.	24
5.2	Transportstacken mellan strategimotor och worker-process.	24
5.3	Sekvensdiagram för felupptäckt och återhämtning.	25

Tabeller

1.1	Producerade dokument	6
5.1	Latensjämförelse mellan in- och out-of-process (μs)	26

Definitioner och akronymer

Nedan hittas en begreppslista med de begrepp, akronymer och definitioner som används i denna rapport.

- **Agentiskt ingenjörskap** - En arbetsmetod där AI-agenter används som ett kompletterande hjälpmedel i utvecklingsarbetet. Utvecklarens roll innefattar här både manuell programmering av komplexa delar, samt att granska kod, guida och verifiera de förslag som verktygen genererar.
- **Backlog** - En lista över uppgifter som behöver genomföras inom ett projekt för att nå det slutgiltiga målet.
- **Backpressure** - Mekanism för att hantera situationer där producenten (en komponent som skickar/genererar data) skriver snabbare än konsumenten läser.
- **Backoff-fördröjning** - En metod för att inte överflöda ett system med anrop som alla misslyckas.
- **Busy-polling** - En process letar efter ny data genom en oändlig loop utan avbrott, ett specialfall av *polling*
- **Cirkulär Buffer** - En begränsad buffert som består av en svans som tillsätter till buffert och ett huvud som tar bort från bufferten.
- **Callbacks** - En metod för att notifiera en viss komponent från en annan komponent i ett system.
- **CPU-pinning** - En teknik för att instruera operativsystemets schemaläggare att en specifik process eller tråd uteslutande ska köras på en förutbestämd processorkärna.
- **Codex/ OpenAI-verktyg** - OpenAI är ett företag som tillhandahåller olika generativa AI tjänster. Codex är en av dessa med fokus på programmering.
- **Discord** - En kommunikationsplattform där man kan kommunicera via meddelanden och röstkanaler.
- **Docker** - Ett containerbaserat virtualiseringsverktyg som paketerar applikationer med alla deras beroenden i isolerade miljöer, vilket gör dem enkla att köra konsekvent på olika system.
- **Docker-container** - Ett lättviktigt, fristående och körbart paket som innehåller allt som behövs för att köra en applikation.
- **Enhetstest** - Testning av enskilda, isolerade delar av källkoden för att verifiera att de fungerar som förväntat.
- **Eventfd** - Ett systemanrop i linux för att skapa en filbeskrivare vars syfte är att låta prenumeranter på *filbeskrivaren* veta när något ändrats.
- **Filbeskrivare** - En unik identifierare som operativsystemet använder som ett handtag för en öppen fil eller I/O resurs.
- **GDB** - Ett verktyg för att debugga C++ kod.
- **Git** - Ett distribuerat versionshanteringssystem som används för att spåra ändringar i källkod och filer, främst inom mjukvaruutveckling.

- **HFT** - Hörfrekvenshandel, en form av algoritmisk handel där datorer köper och säljer finansiella tillgångar på bråkdelar av en sekund.
- **Hot path** - I denna rapport syftar hot path till den biten av kod som tar emot data från marknaderna och ska skicka vidare datan till strategierna, hot path kan även ses som hjärtat av systemet.
- **In-process** - Innebär att en modul laddas och exekveras direkt i huvudapplikationens minnesrymd, samt på samma kärna.
- **IPC** - *Inter-Process Communication*. En samling tekniker och metoder som gör det möjligt för separata operativsystemsprocesser att skicka data och synkronisera sig med varandra.
- **Issue** - Ett ärende i ett versionshanteringssystem (tex *Git*), som används för att spåra uppgifter, buggar eller nya funktioner.
- **JSON** - Textbaserat filformat som ofta används i webbapplikationer för att dela data.
- **Kanbanbräde** - Ett visuellt projektledningsverktyg som används för att strukturera och optimera teamets arbetsflöde.
- **Latens** - Den tidsfördröjning som uppstår från att en händelse sker till att systemet exekverar ett svar.
- **Mirrored cache** - En metod för att dela minne, bland annat mellan processer.
- **Monolitisk (arkitektur)** - Ett system där all kod och alla komponenter körs i samma process och adressrymd utan isolering.
- **Mobbprogrammering** - En agil arbetsmetod där tre eller fler utvecklar samtidigt på samma kodbas. Målet är högre kodkvalitet, minskad komplexitet och bättre kunskaps-spridning.
- **Modern C++** - När modern C++ nämns, menas alla versioner efter C++11 fram till c++-23 (vilket är nuvarande nyaste utgåvan) som möjligtvis har användbar funktionalitet.
- **Multitrådad (arkitektur)** - En exekveringsmodell där ett program kan utföra flera uppgifter parallellt genom att dela upp dem över flera trådar.
- **Mutex** - En synkroniseringsmekanism som säkerställer att endast en tråd åt gången kommer åt en delad resurs.
- **Out-of-process** - När en modul exekveras i en egen operativsystemsprocess, separat från huvudsystemsprocessen. Processen kan även ligga på en egen kärna.
- **Processisolering** - En arkitektur där olika delar av ett program körs i separata processer för att förhindra att en enskild krasch påverkar hela systemet.
- **Polling** - En metod då systemet upprepade gånger läser av statusen från någon enhet.
- **Pull request** - Används inom versionshanteringssystem som Git som ett sätt att föreslå ändringar i koden.
- **Parprogrammering** - En agil arbetsmetod där två personer samarbetar vid en arbetsinstans, en person skriver koden medans den andra personen granskar koden. Dessa roller byts ofta för att öka kunskapsdelning, kodkvalitet och minska buggar.

- **Reaktormönster** - Ett designmönster för nätverksprogrammering där inkommande händelser hanteras sekventiellt i en central loop.
- **Round-trip-latens** - Tiden det tar för ett meddelande eller en signal att skickas från en avsändare till en mottagare och sedan tillbaka igen.
- **RPC** - Remote Procedure Call, Ett protocol som låter systemet göra nätverksanrop som att de vore systemanrop. Självaste nätverksbiten är alltså bortabstraherad.
- **SIGKILL** - Signal i UNIX system (tex Linux) som används för att tvångsavsluta en process.
- **Sprint** - En tidsbegränsad period, under vilken ett tvärfunktionellt team arbetar för att färdigställa en uppgift eller produktinkrement.
- **SPSC** - *Single-Producer Single-Consumer* Datastruktur som har effektiv prestanda i miljöer med delat minne.
- **Strategi** - Handelsalgoritm som tar emot marknadsdata och baserat på denna data tar köp/sälj-beslut.
- **Supervisor** - Komponent som övervakar *worker*-processer, detekterar krascher och initierar omstart.
- **Trusted / Untrusted** - Exekveringsnivåer: *Trusted* = *in-process* (direkt funktionsanrop), *Untrusted* = *out-of-process* (isolerad).
- **Typsäker** - Typsäkerhet handlar om att kompilatorn garanterar att en variabel endast används på det sätt som dess datatyp är avsedd för.
- **UTF-8** - Teckenkodnings protokoll som översätter tecken till unika binära koder på 1-4 byte.
- **Worker** - Separat OS-process som kör en strategi i out-of-process-läge.
- **99:e percentilen** - Det värde under vilket 99 % av alla observerade mätvärden befinner sig under. Endast 1 % av observationerna är högre än detta värde.
- **99,9:e percentilen** - Det värde under vilket 99,9 % av alla observerade mätvärden befinner sig under. Endast 0,1 % av observationerna är högre än detta värde.



1 Inledning

Detta arbete är resultatet av kandidatprojektet *Fault-Tolerant HFT Architecture via Process Isolation*, som genomförts vid Linköpings universitet under våren 2026. Projektets övergripande mål har varit att vidareutveckla en befintlig högfrekvenshandelsmotor (HFT-motor) genom att omforma dess arkitektur från en enkeltrådad *monolitisk* design till en *processisolerad* och mer robust struktur. Arbetet har utförts av en projektgrupp av nio studenter från civilingenjörsprogrammen Datateknik och Mjukvaruteknik.

I detta inledande kapitel presenteras projektets motivation, syfte och frågeställningar som ligger till grund för studien. Kapitlet beskriver även de viktigaste avgränsningarna samt den kontext i vilken projektet har genomförts.

1.1 Motivering

Alexandria Invest utvecklar ett handelssystem som måste exekvera med *latens* lägre än 50 mikrosekunder för att vara konkurrenskraftig på internationella marknader. I det befintliga systemet exekveras samtliga handelsstrategier sekventiellt i samma process, vilket innebär att ett fel i en enskild strategi kan leda till att hela systemet kraschar. Då systemet hanterar betydande kapital utgör denna avsaknad av felisolering en väsentligt operationell risk.

Projektgruppens uppdrag har därför varit att utveckla en lösning där varje handelsstrategi körs i en separat isolerad process. En sådan arkitektur minskar risken för att en enskild strategi påverkar helheten och ökar systemets motståndskraft mot kritiska fel.

1.2 Syfte

Syftet med denna rapport är att besvara de frågeställningar som formuleras i 1.3 samt analysera de erfarenheter som erhållits under projektets genomförande. Erfarenheter har samlats in kontinuerligt under projektets gång och syftar till att bidra till förbättring i både framtida studentprojekt och kundens fortsatta utvecklingsarbete.

Ytterligare är projektets syfte att omstrukturera en monolitisk HFT-arkitektur till att bestå av flera separata isolerade processer och därigenom öka systemets robusthet, felhantering och modularitet.

1.3 Frågeställningar

Under projektarbetet var det gruppmedlemmarnas mening att besvara följande frågor:

1. Hur kan en processisolerad *multitrådad* arkitektur skapa värde för Alexandria Invest AB?
2. Vilka erfarenheter kan dokumenteras från programvaruprojektet *Fault-Tolerant HFT Architecture via Process Isolation* som kan vara intressanta för framtida projekt?
3. Vilket stöd kan man få genom att skapa och följa upp en systemanatomy?
4. Hur kan funktioner i *modern C++* användas för att transformera en befintlig monolitisk HFT-arkitektur till en multitrådad och processisolerad design?

1.4 Avgränsningar

I denna sektion presenteras relevanta avgränsningar kring projektets genomförande.

- Projektet genomfördes inom en ram om 400 timmar per person, i tiden ingick utvecklingsarbete, projektspecifik utbildning, samt dokument- och rapportskrivande.
- Projektgruppens uppgift berör en mycket avgränsad del av ett mycket större system. Således är design och arkitekturbeskrivning avgränsad till de komponenter gruppen har tagit beslut över.
- Systemet utvecklas exklusivt för Linux (Ubuntu 24.04) och ingen hänsyn tas till överflyttningssmöjlighet till andra operativsystem.
- Verifiering sker genom simulerad marknadsdata via tillhandahållna mock-adaptrar. Integration mot live-börser ingår inte i projektets omfattning.
- Levererad produkt bör ses som en Proof of Concept. Syftet är att utvärdera genomförbarhet av processisolering via IPC, inte att leverera en produkt som är redo att köras mot en verklig börs.

1.5 Kontext

Projektet genomfördes inom ramen för kursen TDDD96 vid Linköpings universitet, vilket innebär att arbetet utförts av en grupp studenter i samarbete med en extern kund. Projektet skapade tydliga krav på dokumentation, iterativ utveckling och kontinuerlig kommunikation med kunden.

1.6 Dokumentation

Tabell 1.1 beskriver de dokument projektgruppen producerat under projektets gång.

Tabell 1.1: Producerade dokument

Dokument	Beskrivning
Projektplan	Projektplanen beskriver projektets bakgrund, organisation, mål, dokumentation, utbildning, risker och riskhantering. Projektplanen agerar således som en mall för hur projektet fungerar och är ett styrdokument för projektets organisation.
Kravspecifikation	Beskriver de krav som ställs på den producerade produkten och vad som behöver uppnås för att projektet skall anses vara färdigt. Varje krav presenteras med nummer, version, beskrivning och prioritet.
Kvalitetsplan	Kvalitetsplanen agerar som mall för hur kvalitetsstandarder, processer och metoder ska fungera för att säkerställa att projektgruppens arbete uppfyller de kvalitetskrav som fastställs av målen. Kvalitetsplanen innehåller information kring hur kvalitet mäts och beskriver de kvalitetskontrollerande aktiviteter som ska genomföras.
Arkitekturdokument	Beskriver systemets övergripande arkitektur. Både före och efter att gruppen genomför arbete och strukturerat om den befintliga kodbasen för att fungera efter projektets definierade mål. Arkitekturbeskrivningen underlättar gruppmedlemmarnas förståelse för systemet och ger utvecklare och testare en god översikt över vad målet med koden är.
Testplan	I testplanen beskrivs de tester som ska genomföras, när de ska genomföras och de åtgärder som ska tas ifall dessa misslyckas. Testplanen uppdaterades kontinuerligt för att spegla projektets dåvarande situation gällande utveckling och beslut.



2 Bakgrund

I detta kapitel presenteras en översiktlig beskrivning av kundens och projektgruppens bakgrund kopplat till projektet. Utöver detta behandlas projektets syfte och hur projektgruppen blev tilldelade uppdraget av kunden.

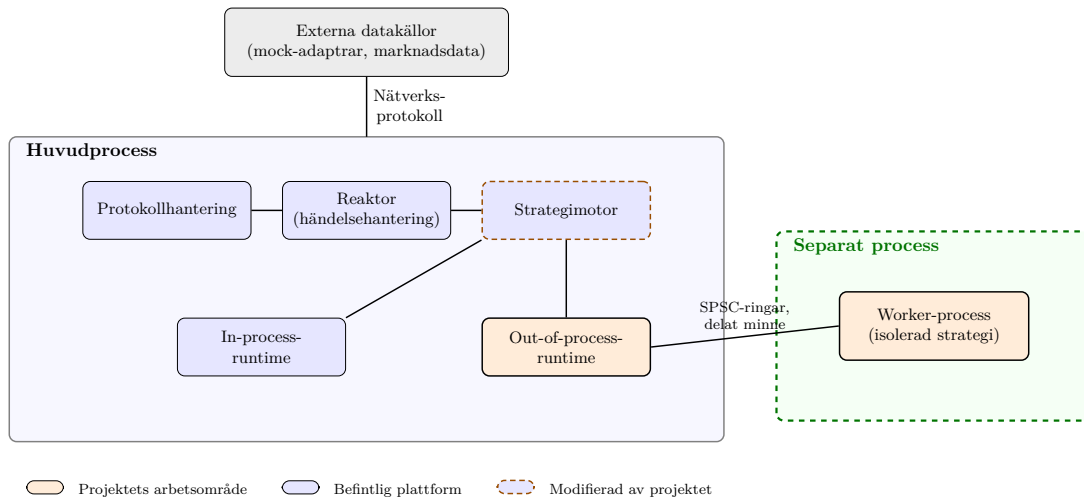
2.1 Kundens bakgrund

Projektets kund är Alexandria Invest AB, vilket är ett mjukvarubolag som utvecklar en plattform för HFT. Som en grundpelare i sin systemarkitektur har företaget ett tekniskt partnerskap med infrastrukturleverantören CryptoStruct [1], på vars teknikstack Alexandria bygger sina kritiska uppkopplingar för marknadsdata och orderläggning. Kärnan i Alexandrias handelsplattform är en intern strategiserver som exekverar olika handelsalgoritmer i realtid. Mjukvaran handlar idag på flertalet internationella marknadsplatser med låga latenstider. Före projektets start var handelsmotorn utvecklad utifrån en rent *in-process* arkitektur, där all marknadsdata, orderläggning och strategiexekvering hanterades i en och samma tråd sekventiellt. Det primära problemet som uppkom med denna in-process arkitektur var avsaknaden av felisolering; ett enda lokalt minnesfel i en spekulativ strategi riskerade att krascha det helhetskritiska systemet.

2.2 Uppdrag

Uppdraget omfattade utvecklingen av en processisolerad arkitektur för att höja systemets robusthet. Alltså göra systemet mer tåligt mot att strategier kraschar och tvingar hela systemet att starta om. Detta görs genom att separera strategiernas miljöer och skapa en felisolerad sandlåda där enskilda komponenter kan återställas vid behov och startas om utan yttre inverkan. Målet är att garantera kontinuerlig drift för den centrala handelsmotorn, oberoende av enskilda strategiers benägenhet att krascha.

Den övergripande systemarkitekturen efter att processisolering har implementerats (det vill säga det nya drifttillståndet), samt projektets tekniska avgränsning gentemot den befintliga plattformen, illustreras i Figur 2.1. Figuren visar att projektet huvudsakligen berör den tillkomna isolerade strukturen för strategikörning (runtime-lagret och *worker*-processen), medan



Figur 2.1: Översikt över arkitektur.

övriga delar av handelsplattformen utgör befintlig infrastruktur. De orangea komponenterna markerar de nya delar som projektet har implementerat för att uppnå processisolering. Blå och grå komponenter tillhandahålls av kunden som befintlig infrastruktur och har inte modifierats.

2.3 Gruppens tidigare erfarenheter

I projektgruppen studerar samtliga medlemmar tredje året på civilingenjörsprogrammen datateknik eller mjukvaruteknik vid Linköpings universitet. Alla medlemmar har sedan tidigare god erfarenhet av att arbeta i mindre projektgrupper från andra kurser i respektive program och har läst "Processprogrammering och operativsystem", vars lärandemål är att studenten ska visa grundläggande kunskaper om bland annat synkronisering, systemanrop och minneshantering [2]. Två gruppmedlemmar är sedan tidigare bekanta med kundens befintliga kodbas.

Med utgångspunkt i erfarenheter från tidigare grupparbeten betonades tidigt vikten av en jämn fördelning av arbetsuppgifter. Syftet var att säkerställa att samtliga gruppmedlemmar bidrar i enlighet med projektets krav. Detta möjliggörs genom tydlig och öppen kommunikation inom gruppen, i kombination med väldefinierade arbetsprocesser för att hantera negativa erfarenheter samt kontinuerligt förbättra arbetsprocessen under projektets gång.



3 Teori

I detta avsnitt presenteras den nödvändiga teori som behövs för att förstå hur projektet fungerar. Relevanta tekniska koncept presenteras tillsammans med de arbetsmetoder och tester som övervägts eller använts under projektets gång.

3.1 Utvecklingsverktyg

Under projektets gång har olika utvecklingsverktyg använts för att organisera och effektivisera utvecklingsarbetet. De mest centrala av dessa presenteras nedan.

3.1.1 C++23

C++23 [3] är den version av C++ utfärdad år 2023. I versionen introduceras ett antal förbättringar som är särskilt relevanta för utveckling av högpresterande och låglatenssystem. C++ är sedan länge etablerat inom områden med hårda realtidskrav, såsom HFT, eftersom språket erbjuder direkt kontroll över minneshantering, trådar, hårdvarunära optimeringar och deterministiska exekveringsmönster.

3.1.2 Git

Git [4] är ett distribuerat versionshanteringssystem som används för att spåra, lagra och organisera förändringar i källkod över tid. Till skillnad från centraliserade system lagrar Git en komplett kopia av hela versionshistoriken på varje utvecklarens dator, vilket gör arbetet mer robust och mindre beroende av en central server. Detta möjliggör parallellt arbete, återställning av tidigare versioner och effektiv hantering av konflikter när flera utvecklare arbetar på samma kodbas.

3.1.3 GitLab

GitLab [5] är webbaserad tjänst som hjälper utvecklare hantera källkod. GitLab kan även användas för att organisera utvecklingsprocessen, bland annat genom att använda sig av *issues*. En issue kan sedan länkas till ändringar i koden vilket tydliggör kodändringar. Vidare finns funktionalitet för versionshantering av koden vilket är i samband med Git.

3.1.4 Docker

Docker [6] är ett projekt som används för att distribuera program i bärbara och självförsörjande *Docker-containrar*. Docker tillåter användare att paketera applikationer till standardiserade enheter. Detta agerar som en brygga mellan utvecklarens system och det system programmet senare ska köras på.

3.2 Arbetsmetoder

I detta avsnitt presenteras teorin bakom de arbetsmetoder som har använts under projektets gång.

3.2.1 Kanbanbräde

Ett *Kanbanbräde* är ett visuellt verktyg som används för att planera, följa upp och organisera arbete i ett projekt. Brädet är uppdelat i olika kolumner. Varje kolumn representerar ett skede i arbetsflödet, exempelvis arbete som ska påbörjas, aktiva arbetsuppgifter och avslutade arbetsuppgifter. I kolumnerna finns det kort som beskriver arbetsuppgifter, dessa kan markeras som klara alternativt flyttas mellan kolumner i takt med att arbetet fortskrider. På så sätt skapas en tydlig överblick över arbetsflödet och den aktuella arbetsbelastningen.

3.2.2 Scrum

Scrum [7] är ett agilt arbetssätt inom mjukvaruutveckling som används för att strukturera och effektivisera projektarbete. Arbetet delas upp i sprintar där teamet fokuserar på att utveckla och leverera en avgränsad del av produkten. Inför varje sprint planeras vilka uppgifter som ska genomföras utifrån projektets prioriteringar. Under sprinten följs arbetet upp via kontinuerliga möten. Efter sprintens slut utvärderas resultatet och arbetsprocessen. Utvärderingen lägger grunden till nästkommande sprint.

Backlog En Scrum *backlog* [8] är en lista över arbete som behöver genomföras inom ett projekt för att nå det slutgiltiga målet. Backloggen används som ett planeringsverktyg där teamet kontinuerligt uppdaterar, förtydligar och omprioriterar innehållet utifrån projektets behov. Inför varje sprint väljs de uppgifter från backloggen som bedöms vara viktigast och möjliga att genomföra under perioden. För att organisera en backlog finns det flera metoder, där användningen av ett Kanbanbräde (se avsnitt 3.2.1) är ett vanligt exempel.

Mål Inom *Scrum* används främst två typer av mål, den första typen av mål är långsiktiga mål, dessa innefattar stora delar av produktens design och utveckling. Det andra målet är sprint-mål, dessa sätts upp i början av varje sprint och målbilden är att klara av dessa mål tills nästa sprint. Tanken är att långsiktiga mål ska brytas ner i sprint-mål.

Organisation Organisationen inom *Scrum* bygger på tydliga roller och nära samarbete inom teamet. Vanligtvis består organisationen av en produktägare [9], ett utvecklingsteam och en *Scrum-master* [10]. Produktägaren ansvarar för att prioritera arbetet och säkerställa att utvecklingen som görs är relevant. Utvecklingsteamet består av utvecklare [9] och genomför det praktiska arbetet och ansvarar gemensamt för att nå sprintens mål. *Scrum-master* stödjer teamet genom att underlätta arbetsprocessen och se till att processer inom *Scrum* genomförs på rätt sätt. Idén bakom organisationen är att främja ansvarsfördelning och kommunikation samt att gruppen ska kunna jobba på ett flexibelt och anpassningsbart sätt.

3.2.3 Agentiskt ingenjörskap

Agentiskt ingenjörskap rör metodiken att använda artificiell intelligens, specifikt Stora Språkmodeller (LLM:er), som autonoma agenter i programvaruutvecklingen [11]. Istället för att använda AI enbart som ett verktyg för auto-completion eller så kallat *vibe-coding* [12] där systemet avslutar korta loopar eller metoder, innebär agentiskt ingenjörskap att agenten matas med stor strukturell systemkontext, alltså med tusentals rader kod. Agenten får sedan i uppdrag att iterativt planera, generera och testa hela initiala arkitekturutkast. Utvecklarens roll skiftar därmed från att primärt skriva syntax till att som en "arrangör" [13] arbeta med *prompt engineering*, kontextinjektion, kodgranskning samt att integrera de genererade systemdelarna mot existerande domänlogik.

3.2.3.1 Lärande och användande av AI-assistenter

Något som har diskuterats vid användande av LLMs som stöd, är att det blir en kognitiv avlastning, vilket sparar tid och energi för en erfaren utvecklare. Men för någon som inte tidigare lärt sig alla steg i en process och saknar domänkunskap kan detta innebära att man helt enkelt missar vissa steg, och det blir svårt att se när AI gör fel [14]. Shen och Tamkin har i *How AI Impacts Skill Formation* identifierat sex olika mönster som används vid programmering med AI-assistenter, där det är tydligt att de som avlastar en stor del av arbetet till assistenten gör det på en stor bekostnad av lärande[15]. De som däremot tar sig tid att läsa koden och försöka förstå varje rad, eller använder AI-assistenten för att ställa frågor om genererad kod eller ramverk presterade betydligt bättre i uppföljande quiz i studien (24-39% korrekta svar i den låg engagerade gruppen jämfört med 65-86% för hög engagerade gruppen)[15].

3.3 Trådar, processer och I/O

I detta kapitel beskrivs teorin för de olika datastrukturer som användes i implementationen av projektet.

3.3.1 Atomiska operationer

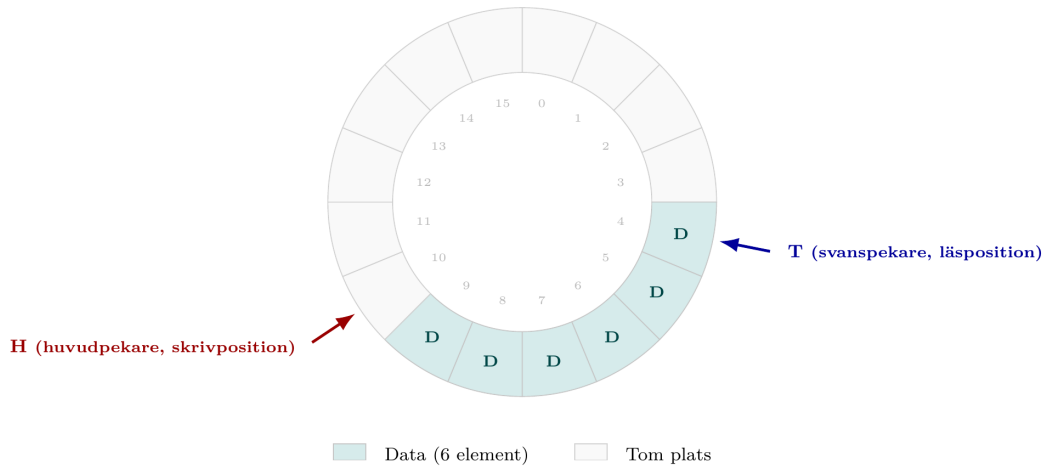
Inom programmering är en atomisk operation [16] en instruktion som utförs på en enkel variabel helt igenom eller inte alls. Denna instruktion körs utan att andra processer kan se eller påverka operationen medan den pågår.

Atomiska operationer förhindrar därmed problem såsom kapplöpningsvillkor som kan uppstå i flertådiga system, där till exempel en tråd försöker använda en enkel variabel som inte har bearbetats klart av en annan tråd. Till skillnad från lås gäller atomiska operationer enbart för enkla variabler och har oftast bättre prestanda än låsbaserade implementationer.

3.3.2 SPSC-Ring struktur

En SPSC-ring [17], *Single-Producer Single-Consumer* ring, är en implementation av en *cirkulär buffert* som följer begränsningarna, enbart en tråd eller process kan skriva in i bufferten och enbart en tråd eller process kan hämta data från bufferten. Instansen som skriver till bufferten har en pekare till utrymme som kallas för huvudpekaren. Instansen som läser data från bufferten har en pekare till äldsta använda utrymme som kallas för svansen. När en instans vill skriva data till ringen skrivs data till det utrymme som huvudpekaren pekar på, även om det redan finns data där. Huvudpekaren flyttas sedan framåt. När en instans vill läsa data i ringen läses data från utrymme som svanspekaren pekar på. Svanspekaren flyttas sedan framåt. När en pekare i en cirkulär buffert når slutet av bufferten kommer pekaren att återgå till början av bufferten. Om någon av pekarna förflyttar sig så att de befinner sig i samma utrymme

anses bufferten vara tom. Om huvudpekaren befinner sig en utrymme bakom svanspekaren är bufferten full. Observera figur 3.1 där en grafisk demonstration av en SPSC-ring demonstreras.



Figur 3.1: SPSC-ringbuffert (*Single-Producer Single-Consumer*)

3.3.3 Låsfritt system

Att en implementation är låsfri [18] innebär att systemet garanterar att åtminstone en process kommer att lyckas med att slutföra en operation inom en ändlig mängd tid. En låsfri implementation använder sig av atomiska operationer för att exekvera instruktioner. Ett låsfritt system har inte problem med dödläge, där två eller fler trådar eller processer väntar på varandra att släppa en resurs, som kan uppstå i en låsbaserad implementation.

3.3.4 Skillnad i Processer och trådar

Skillnaden mellan en tråd och en process är minneshantering [19, pp. 106–107, pp. 160–161]. En process är isolerad från övriga datorn och innehåller sin egna sektion av minnet. Alltså kan andra processer inte påverka de element placerade i detta minne. En tråd kan ses som en underprocess som är ansvarig för en del av systemet inom processen. En process kan ha flera trådar och då de är placerade inom samma minnesrymd kan de fritt komma åt varandras minne.

3.3.5 Epoll

Epoll [20, pp. 1355–1358] är en I/O händelsenotifieringsmekanism som används för att övervaka flera filbeskrivare och notifierar vilka som är redo för I/O. En epoll instans består av två listor, en intresselista och en redolista. Intresselistan är listan som är ansvarig för att innehålla filbeskrivarna som ska övervakas. Redolistan är listan av alla filbeskrivare som är redo för I/O som också befinner sig i intresselistan.

3.4 High Frequency Trading (HFT)

HFT är en teknik inom börshandel där köp- och säljordrar exekveras varje mikrosekund, med målet att generera en liten vinst per order som ackumuleras över tid till betydande vinster [21]. Framgång med denna handelsmetod kräver tre viktiga faktorer. Den första faktorn, som även är den mest betydelsefulla, är att ta emot information från marknaden före andra aktörer via fiberkablar som kan överföra information nära ljusets hastighet och hårdvaruinfrastruktur

placerad i direkt fysisk närhet till börsens servrar [22]. Den andra faktorn är smarta handelsstrategier byggda med programvara som kan analysera inkommande marknadsdata och fatta beslut om köp- och säljorder i realtid [23].

3.5 Kommunikation mellan processer (IPC)

Interprocesskommunikation (IPC, från engelskans *Inter-Process Communication*) [24] definierar de tekniker, strukturer och protokoll som tillåter separata processer i ett operativsystem att överföra data och synkronisera sina exekveringar. Som grundregel tillhandahåller operativsystemet en strikt minnesisolering mellan processer för att förhindra att ett fel i en process korrumperar en annan. Problemet som IPC löser är att tillåta säkert datautbyte trots denna isolering.

Det finns flera olika etablerade metoder för att åstadkomma IPC, med varierande egenskaper gällande prestanda och användbarhet:

- **Nätverks-sockets och UNIX-domainsockets:** En flexibel modell som tillåter kommunikation både lokalt och över nätverk, men som medför betydande latens och indirekta kostnader (overhead) eftersom varje meddelande måste passera operativsystemets kernel.
- **Pipes:** En enkel enkelriktad dataström som lämpar sig väl för sekventiella flöden mellan föräldra- och barnprocesser.
- **Delat minne:** Det mest högpresterande alternativet där två eller flera processer ges direktåtkomst till samma fysiska minnessegment. Eftersom data kan läsas och skrivas direkt utan att kopieras genom operativsystemets kernel minimeras latensen, vilket gör metoden särskilt lämpad för prestandakritiska realtidssystem. Synkronisering måste dock hanteras manuellt via exempelvis semaforer, *mutexer* eller låsfria ringbuffertar.

Eftersom projektet syftar till att isolera strategier i separata processer, måste all kommunikation ske via en IPC-mekanism. Eftersom processisolering skyddar minnesrymder men skapar indirekta kostnader, påverkar valet av IPC-lösning systemets totala latens avsevärt. För att möta de strikta latenskraven inom HFT föll valet i detta projekt på delat minne i form av låsfria cirkulära *SPSC*-ringbuffertar, vilket eliminerar kernel-overhead vid dataöverföringen.



4 Metod

I detta kapitel beskrivs de metoder som använts under projektets gång. Kapitlet beskriver även de metoder som använts för att besvara frågeställningarna från avsnitt 1.3.

4.1 Värde för kunden

För att skapa värde för kunden ska gruppen följa kraven som kunden sätter på produkten. Det är även viktigt att gruppen presentera hur produkten möter kraven på ett rimligt sätt. Kunden tillhandahållas även relevanta interna dokument, till exempel kravspecifikation och projektplan för att kunna ge synpunkter och önskemål på hur gruppen arbetar och vilken funktionalitet som implementeras samt hur den implementeras. För att säkerställa godtagbar kvalitet på koden, genomförs testing, denna beskrivs i avsnitt 4.3. Gruppen ska också bekräfta värdet genom att tillhandahålla prestandamätningar för latens i koden, genomförandet beskrivs under avsnitt 4.3.2.

Att maximera värde för kunden är övergripande för hela projektet, därför är övriga metodval som behandlas mer noggrant i resterade avsnitten under kapitel 4 också relevant för att svara på denna första frågeställningen beskriven i avsnitt 1.3.

4.2 Metodval

Detta avsnitt lyfter de aktuella metodvalen.

4.2.1 Arbetsmetodiken som användes

Arbetsmetoden som användes under projektets gång är baserad på arbetsmetoden scrum. Gruppens arbetsmetod bestod av att dela upp arbetet i tre huvudsakliga förbättringsområde, dessa är beskrivna som tre olika issues som går att läsa om i avsnitt 5.2.2. Utifrån dessa issues delades gruppen in i mindre arbetsgrupper med tre personer vardera. Därefter var det fritt upp till arbetsgrupperna att bestämma tid och plats för att arbeta vidare på tilldelat issue.

4.2.2 Planeringsmöte

Under mötet planeras nästkommande sprint [25]. Här sker det en diskussion inom gruppen om vilka uppgifter som ska genomföras och arbetsuppgifter delas upp. Mötet leds av *Scrum-master*. Målet med mötet är att alla gruppmedlemmar ska vara uppdaterade på de nya arbetsuppgifterna och att det ska finnas en möjlighet för medlemmarna att komma med önskemål och tankar om vad som behöver göras härnäst. Under mötet kommer projektets Kanbanbräde uppdateras.

4.2.3 Utvärderingsmöte

En *sprint-review* [26] hålls under en pågående sprint, förslagsvis halvvägs under den pågående sprinten. Inför mötet ska kvalitetssamordnaren skickat ut en enkät där varje gruppmedlem får beskriva och evaluera hur den nuvarande sprinten går. Kvalitetssamordnaren sammanställer sedan informationen. Under mötet presenterar kvalitetssamordnaren informationen för gruppen och inleder en diskussion tillsammans med *Scrum-master* om ett Sprint-mål behöver revideras. I samband med mötet finns möjlighet för gruppens medlemmar att ta upp önskemål och tankar om processerna som används inom projektet.

4.2.4 Issues som kanban

Gruppen valde att hantera arbetsuppgifter som issues via Git. Varje nytt identifierat problem dokumenterades som en ny issue. Varje problem där den uppskattade tidsåtgången var större än 10 timmar ska bli uppdelat som en egen issue. För att säkerställa kvalitet och enhetlighet i projektledningen tog den konfigurationsansvarige fram en standardiserad mall för hur dessa issues skulle utformas.

4.2.5 Överlämning av kodbas

Genom automatisk och manuell testning säkerställer gruppen att koden som finns på den primära branchen är redo för fortsatt utveckling efter att överlämnandet av kodbasen har skett. I denna rapport lyfter vi också en rekommenderad väg framåt för kunden, så att fortsatt arbete kan påbörjas direkt med en tydlig riktning.

4.2.6 Användning av agentiskt ingenjörskap

När utvecklingsarbetet skulle påbörjas efter att utbildningsperioden var färdig insåg gruppen att utbildningen inte visade sig vara tillräcklig för att effektivt kunna arbeta i kodbasen mot projektets mål. För att komma förbi detta initiala dödläge togs beslutet att komplettera gruppens egna manuella utvecklingsarbete med agentiskt ingenjörskap [11] som ett komplementärt hjälpmedel. AI-verktygen användes restriktivt och inte särskilt liberalt; de tillämpades främst i den tidiga fasen för att snabbare bygga en förståelse för kodbasen och ta fram enstaka initiala arkitektoniska skelett, vilka gruppen sedan manuellt vidareutvecklade, integrerade och förfinade. Den övervägande majoriteten av kodbasen skrevs för hand av gruppmedlemmarna under hela projektets gång, medan AI-verktygen fungerade som ett bollplank och punktvis stöd.

4.2.6.1 Verktyg

Två AI-verktyg användes som komplement till gruppens egna utvecklingsarbete:

- **OpenAI Codex** [27] – Ett terminalbaserat agentverktyg som exekveras lokalt via kommandoraden. Codex gavs läsätkomst till projektets Git-repository och användes både

via dess CLI-gränssnitt och via den tillhörande desktopapplikationen. Verktöget nyttjades restriktivt, främst för att utforska specifika lösningsalternativ och generera enstaka initiala kodförslag som utvecklarna sedan bearbetade manuellt.

- **Google Gemini** – Användes dels via dess CLI-verktyg med kodbasen som kontext, dels via webbgränssnittet. Gemini nyttjades i huvudsak som diskussionspartner vid arkitekturbeslut och för att utforska designalternativ.

Båda verktygen gavs åtkomst till hela kodbasen som arbetskontext. Eftersom den befintliga kodbasens dokumentation och filstruktur konsekvent pekade mot relevanta moduler, kunde agenterna under gruppens styrning lokalisera de filer som berörde den aktuella uppgiften utan att gruppen manuellt behövde begränsa kontexten. Detta underlättade gruppens arbete med att ställa precisa frågor och få relevanta svar baserade på kodbasens faktiska struktur.

4.2.6.2 Promptstrategi

När AI-verktygen användes strukturerades prompterna utifrån projektets kravspecifikation och den framtagna systemanatomien. Arbetsflödet där AI-stöd tillämpades följde en iterativ process:

1. **Utforskande och utkast:** I den inledande fasen användes AI-verktygen vid enstaka tillfällen för att utforska möjliga arkitekturer och generera grova utkast. Gruppen formulerade uppgifter på en hög abstraktionsnivå, exempelvis att föreslå ett IPC-lager baserat på SPSC-ringbuffertar. Resultaten utgjorde endast ett diskussionsunderlag som gruppen gemensamt granskade, kasserade eller skrev om manuellt från grunden.
2. **Manuell förfining:** Den övervägande delen av varje delsystem implementerades manuellt av gruppen. AI-verktyg användes sparsamt, huvudsakligen som stöd för att generera enkla mallar eller lösa isolerade, repetitiva uppgifter. Gruppmedlemmarna skrev själva all prestandakritisk kod och alla delar som krävde djup förståelse för systemets sammanhang.
3. **Integration och test:** All kod, oavsett om den skrivits manuellt eller med AI-stöd, integrerades med den befintliga kodbasen och verifierades genom de testmetoder som beskrivs i avsnitt 4.2.6.3.

En central begränsning i arbetsflödet var att AI-verktygen inte fick modifiera befintlig kod i kundens kodbas. Ny funktionalitet implementerades i separata filer och moduler som integrerades med det befintliga systemet via dess existerande gränssnitt. Denna begränsning minskade risken att kod oavsiktligt korrumpade befintlig funktionalitet.

Strategin för hur prompterna hanterades formades av de studier som refereras i avsnitt 3, i synnerhet kring riskerna med kognitiv avlastning [14] och de engagemangsmönster som Shen och Tamkin identifierat [15]. Gruppen eftersträvade konsekvent det aktiva engagemanget, där varje genererad kodsutt lästes, ifrågasattes och förstods innan den accepterades.

4.2.6.3 Verifiering av kod

Oavsett om kod skrivits manuellt eller med stöd av AI-verktyg tillämpades samma verifieringsprocess. I ett latenskritiskt system ställs höga krav på korrekthet, och följande steg genomfördes systematiskt:

1. **Manuell kodgranskning i arbetsgruppen:** All ny kod granskades gemensamt av den arbetsgrupp om tre personer som tilldelats det aktuella issuen. Granskningen fokuserade på att varje gruppmedlem förstod kodens logik och kunde motivera varför den var korrekt. Kod som inte förstods av samtliga medlemmar i arbetsgruppen accepterades inte.
2. **Automatiserade tester:** Efter att koden passerat den manuella granskningen exekverades den mot projektets testsvit, beskriven i avsnitt 4.3.4. Testfallen inkluderade både positiva tester för standardflöden och negativa tester med avsiktligt felaktig indata, exempelvis kraschande strategier och fulla ringbuffertar.
3. **Formell kodgranskning via merge request:** Innan kod integrerades i huvudgrenen krävdes godkännande från minst två gruppmedlemmar utanför den ursprungliga arbetsgruppen, i enlighet med den process som beskrivs i avsnitt 5.2.2. Detta skapade en ytterligare verifieringsbarriär där kod granskades av personer som inte varit delaktiga i genereringen.
4. **Systemintegrationstest:** Avslutningsvis kördes det fullständiga systemintegrations-testet med mock-adaptrar för att verifiera att den nya koden fungerade korrekt i samverkan med den övriga arkitekturen.

Sammanfattningsvis användes AI-verktygen som ett komplement till gruppens egna utvecklingsarbete, inte som en ersättning. Gruppmedlemmarna skrev, läste och modifierade kod under hela projektet. AI-verktygen bidrog främst med att snabba upp den initiala utforskningen och minska tröskeln för att komma igång i en komplex kodbas. Kvalitetsgarantin låg i det gemensamma skyddsnätet av granskning och testning som tillämpades på all kod, oavsett ursprung.

4.2.7 Insamling av erfarenheter

För att kontinuerligt dokumentera och samla de erfarenheter som erhöles under projektets gång användes följande metoder:

- Veckovisa enkäter: Kvalitetssamordnaren skickade löpande ut enkäter där gruppmedlemmarna systematiskt fick utvärdera sprinten och skriftligt reflektera över processer och tekniska utmaningar.
- Veckovisa möten: Den insamlade enkätdata analyserades och diskuterades därefter gemensamt under återkommande utvärderingsmöten för att identifiera förbättringsområden.
- Spårbarhet via versionshantering: Diskussionsunderlag och beslut kring kod och arkitektur dokumenterades kontinuerligt via issues i GitLab, vilket utgjorde en logg över gruppens tekniska lärdomar och vägval.

4.2.8 Systemanatomi

Följande sektioners syfte är att förklara hur gruppen dragit slutsatser om befintliga kopplingar och flöden mellan komponenter i systemet, men även hur framtida kopplingar och flöden måste vara för att de kraven som givits ska uppfyllas.

4.2.8.1 Skapande av systemanatomi

Arbetet inleddes med individuella litteratur- och kodstudier enligt utbildningsplanen (beskriven i avsnitt 4.5) för att förstå systemets uppbyggnad. Därefter kartlade gruppen gemensamt

hur programmet fungerade. Detta gjordes genom att varje gruppmedlem presenterade sitt perspektiv kring hur systemet är uppbyggt följt av en diskussion då gruppmedlemmar tänkte olika.

För att verifiera de visuella modellerna användes stegvis felsökning med hjälp av *GDB*. Genom att exekvera koden interaktivt kunde gruppen spåra dataflöden och signalering i realtid, vilket var avgörande för att identifiera kritiska områden såsom *IPC*-lagret och användningen av låsfria *SPSC*-köer.

För att effektivisera och underlätta analysarbetet användes de generativa AI-verktygen *Codex* och *Gemini*. Dessa användes främst för att extrahera strukturell information ur källkoden och generera preliminära diagram över systemets uppbyggnad. Detta gjordes genom att ge Codex läsrättigheter över hela kodbasen, där prompterna var i stil med 'förklara hur systemet fungerar'. Valet att använda dessa verktyg skedde med anledning av kundens omfattande och komplexa kodbas i kombination med projektets begränsade tid. Bedömningen var att en förståelse av tillräckligt djup över kodbasen skulle ta för stor andel av budgeten. De val som gjordes under anatomiutformningen prioriterades utifrån de strikta latenskrav som definierats i projektets kravspecifikation.

4.2.8.2 Uppföljning av systemanatomi

Gruppen använde systematiskt systemanatomin under projektets skede med följande metoder:

- **Identifiera problematik:** Anatomin granskades för att lokalisera specifika områden där logik saknas för att bedriva multiprocess arkitektur med låg latens.
- **Kartläggning av beroenden:** Systemets komponenter analyserades för att identifiera vilka delar som hade minst beroende av övriga moduler. Ett teoretiskt exempel på ett sådant beroendeflöde illustreras i 4.1, där komponent 1 är oberoende av de efterföljande.



Figur 4.1: Linjärt beroendeflöde mellan komponenter

- **Fördelning av parallellt arbete:** Utifrån kartläggningen av beroenden delades den nio personer starka projektgruppen in i mindre arbetsgrupper. Systemanatomin användes här konkret för att tilldela ansvarsområden som kunde utvecklas parallellt utan att grupperna skulle störas av varandra.

Kvalitetssamordnaren genomförde även enkäter där gruppmedlemmarna beskrev hur de använde systemanatomin, vilket sedan kartlades under utvärderingsmötena.

4.3 Testning

Produktens testning genomfördes i tre moduler, dessa går igenom under följande kapitel.

4.3.1 Funktionalitet

Första modulen är funktionalitetsmodulen med fokus på verifiering av kundens funktionella krav. Följande tester gjordes i funktionalitetsmodulen för att produkten skulle klassificeras som

funktionell. Det ska vara möjligt att exekvera strategier i både *in-process* och *out-of-process* lägen; Där en kraschad strategi inte ska orsaka avbrott i systemet, och strategier ska exekveras parallellt utan risk att påverka varandra. Verifieringen av dessa kärnfunktioner utfördes genom växling mellan de två lägena samt genom exekvering av en avsiktligt korrupt strategi för att analysera systemets robusthet mot fel.

4.3.2 Hur tester användes för prestandamätningar

Den andra modulen är prestandamodulen vars syftet är att kontrollera huruvida system har tillfredsställande *round-trip-latens*, återhämtningstid och strategi-isolering. Prestandatesterna genomfördes genom att mäta exekveringstiden för moduler som implementerar dessa funktioner, beskrivit vid avsnitt 4.3.2. Kravet för strategi-isolering behandlar strategimotorns status efter en strategikrasch. En felaktig strategi som försöker initiera en otillåten minnesåtkomst kördes på arkitekturen för att testa om motorn överlever krascher och startar om de kraschade strategierna.

4.3.2.1 Latenskostnad

I projektet definieras latens som den tur-och-retur-latens som uppstår när ett meddelande passerar IPC-lagret. Detta innefattar den tid det tar att överföra data via *SPSC*-ringbuffertar i delat minne från huvudmotorn till den isolerade worker-processen, samt att returnera ett svar, i förhållande till exekveringstiden för ett direkt funktionsanrop i en monolitisk *in-process*-arkitektur. Se fig. 5.1 för en beskrivning av worker-processerna samt hur komponenterna interagerar.

4.3.2.2 Genomförande

För att testa prestanda testades systemet i två delar. Först mättes tiden för att ta emot marknadsdata i form av ett *UTF-8*-formaterat *JSON*-meddelande på 111 byte från transportlagret (visualiserat i fig. 5.2), samt att skicka denna data till en strategi. Detta test gjordes enbart för *in-process* och ger en bild av systemets totala latens.

Därefter testades *Round-trip-time* mellan två processer för en komplett handelsorder. I föräldraprocessen serialiseras en C++ struct (innehållande `std::string` och `std::vector`) till ett *JSON*-paket på 481 byte, som sedan skrivs till ringbufferten i delat minne, varpå arbetsprocessen väcks. Arbetsprocessen väntar på en signal, läser in datapaketet från ringbuffern, deserialiserar *JSON*, serialiserar det igen, skriver in svaret på den andra ringbuffern och väcker föräldraprocessen. Föräldraprocessen läser då av svaret och stoppar mättiden. Serialiseringskostnaden beror på meddelandets storlek och nästlingsdjup, varför de två testerna med olika paketstorlekar (111 respektive 481 byte) ger en bild av hur latensen skalas. Det senare testet används för att se hur projektets implementation påverkar latens då det är isolerat till *IPC:n*.

4.3.2.3 Mätning av robusthet och återhämtningstid

För att mäta systemets återhämtningstid vid en strategikrasch utfördes följande procedur. Först startades en arbetsprocess i *out-of-process*-läge och systemet verifierades vara i stabilt drifttillstånd. Därefter skickades signalen *SIGKILL* till arbetsprocessen, varpå mättiden startades. Strategimotorn detekterar att den underordnade processen avslutats och initierar en omstart. Mättiden stoppades när den nya arbetsprocessen signalerade att den var redo att ta emot meddelanden. Denna procedur upprepades ett flertal gånger för att erhålla ett statistiskt underlag för återhämtningstiden.

4.3.3 Integration

Den tredje modulen är integrationsmodulen där det säkerställs att projektets tillägg till det övergripande systemet inte påverkar andra delar på ett sätt som försämrar deras funktionalitet. En omfattande testfil bestående av ett stort antal testfall användes för att verifiera integrationen av ny mjukvara i det befintliga systemet. Testet involverar alla komponenter som arkitekturen består av, vilket inkluderar strategimotorn, en kompetent handelsstrategi, simulerad marknadsdata och mätningsverktyg som loggar resultat.

4.3.4 Riktlinjer för modul- och enhetstestning

Testningsstrategin för varje modul är densamma. Inledningsvis genomförs positiva tester där standardprocedur undersöks för att verifiera att de grundläggande kraven uppfylls. Sedan testas hur modulen reagerar på felinmatningar (negativa tester) för att undersöka hantering av felaktig indata. När modulen bedöms fungera korrekt och inget oönskat beteende uppstår, påbörjas utforskande tester i syfte att utvärdera systemets effektivitet och beteende i mer komplexa scenarier.

Varje testfil följer samma mall för att garantera god kvalitet och tydliga acceptanskriterier. Filerna inleds med kommentarer som innehåller information gällande testets omfattning och instruktioner för hur testet körs. Efter det definieras en testrigg vilket innehåller makron för skapandet av testfall. Testning av mindre komponenter eller ändringar underlättas med riggen eftersom den tidskrävande processen att verifiera korrekthet automatiseras. Slutligen skrivs själva testfallen med alla lokala variabler och förväntade värden av slutresultatet. Projektets slutliga version omfattade 30 testfiler där varje testfil exekverar cirka tio testfall. En rimlig uppskattning för det totala antalet testfall som undersöktes under projektets gång är därför approximativt 300 testfall.

4.4 Roller

I projektets organisation har vi delat upp gruppen i olika ansvarsområden för att tydligt definiera vad varje person har som ansvarsområde. Rollerna är presenterade nedan.

- **Teamledare:** Ansvarig för att driva arbetet framåt, planerar och koordinerar arbetet. Ser till att arbetsprocesser följs och håller gruppens kanbanbräde uppdaterat. Teamledaren har utslagsröst då beslut skall tas och är gruppens representant utåt.
- **Dokumentansvarig:** Huvudansvarig för att säkerställa att projektdokument produceras och hålls uppdaterade. Ansvarar också för att kvalitetssäkra dokumenten.
- **Analysansvarig:** Har ansvar för kravspecifikationen och är gruppens kontaktperson till kunden. Översätter kundens önskemål till krav.
- **Arkitekturansvarig:** Ansvarar för de övergripande designval som gruppen gjort under projektets gång. Ser till att det som produceras är enhetligt och väl strukturerat.
- **Konfigurationsansvarig:** Har ansvar över projektets Git-repo och är ansvarig för att versionshanteringen följer standardiserade processer och strategier kring branching och merging. Är också ansvarig för att ändringar i systemet är spårbara och för att byggmiljöer samt nya releaser hanteras på ett säkert sätt.
- **Kvalitetssamordnare:** Har huvudansvar för projektets övergripande kvalitet. I detta ingår att skriva kvalitetsplan och planera utvärderingsmöten, samt inhämta och analysera gruppens åsikter om den nuvarande processen.

- **Testledare:** Har ansvar över att skriva testplanen och att planera testningen av systemet.
- **Utvecklingsledare:** Har ansvar för att organisera utvecklingsarbetet. Ser till att arbetet kontinuerligt går framåt och har koll på vad nästa steg i utvecklingen är.
- **Integrationsspecialist:** Har expertkunskap kring den befintliga kodbasen och agerar kunskapsbärande och tekniskt bollplank gällande de tidigare implementationsvalen som gjorts i den givna koden. Ansvarar också för att gruppens arbete ska fungera i kontext av den befintliga kodbasen och sömlöst bli en del av den.

4.5 Utbildning

Den befintliga kodbasen är skriven i C++23 och bygger på ett antal avancerade koncept inom systemprogrammering som inte behandlas i grundutbildningens obligatoriska kurser; Låsfri samtidighet, IPC, och händelsestyrt *reaktormönster* är exempel på kunskapsområden som projektmedlemmarnas utbildning inte täcker. För att säkerställa att samtliga gruppmedlemmar kunde bidra till utvecklingsarbetet på ett meningsfullt sätt prioriterades en utbildningsfas tidigt i projektet. Projektets integrationsspecialist, som sedan tidigare hade erfarenhet av den befintliga kodbasen, ansvarade för att planera utbildningen. Utbildningsplanen bestod av två huvudsakliga delar: en teoretisk genomgång av centrala koncept samt en uppsättning praktiska programmeringsövningar.

4.5.1 Teoretisk genomgång

En skriftlig genomgång togs fram med syfte att ge gruppens medlemmar den grundförståelse som krävdes för att kunna förstå och modifiera den befintliga koden. Dokumentet behandlade följande ämnesområden, i en ordning som utgick från grundläggande begrepp och successivt byggde mot de mest projektspecifika:

- **Abstrakta klasser och virtuella metoder** – Undervisningen fokuserade på hur systemmallar byggs upp och hur gruppen kunde tillämpa polymorfism för att hantera underklasser via gemensamma gränssnitt.
- **Stacken och heapen** – Momentet behandlade skillnaden i allokeringstyper. För att konkretisera varför och hur dynamisk allokering undviks i latenskritisk kod.
- **Cacheminne** – Här studerade gruppen hur datorns cacheminne fungerar och varför sättet data struktureras i minnet har stor påverkan på systemets hastighet.
- **Låsfri programmering** – Genomgången fokuserade på hur atomiska variabler kan tillämpas för att undvika lås, specifikt anpassat för kodbasens delade *SPSC*-köer.
- **Processer och trådar** – Denna del av utbildningen syftade till att förankra förståelsen för projektets mål om processisolering, genom att studera skillnaderna mellan delade och separata adressrymder och hur dessa påverkar ett systems feltolerans.
- **Händelsestyrd I/O och epoll** – Ett moment som behandlade hur den befintliga systemet använder *epoll* för att hantera flera händelser samtidigt utan att programmet stannar upp.

Genomgången avslutades med en detaljerad läsguide som pekade ut specifika filer och radnummer i kodbasen som gruppmedlemmarna förväntades studera. Tonvikten låg på de filer som direkt berörde projektets arbete, framför allt strategimotorns interface, runtimeimplementationer och kommunikationsprimitiver.

4.5.2 Praktiska övningar

Utöver den teoretiska genomgången sammanställdes en lista med programmeringsövningar hämtade från kurserna TDDE18 Grundläggande programmering i C++, och TDDD38 Avancerad programmering i C++ vid Linköpings universitet, samt ett urval av algoritmiska problem. Övningarna valdes för att täcka de koncept som var mest relevanta för projektet och prioriterades i tre nivåer.

4.5.3 Hur modern C++ ska användas

Gruppen tillägnar sig huvudsakligen kunskap om moderna C++-funktioner genom intern utbildning, men även genom diskussioner där generativ AI agerar bollplank.

För att utvärdera i vilken utsträckning dessa moderna metoder faktiskt tillämpas i projektet, genomförs en manuell kodgranskning. Denna granskning är av kvalitativ karaktär, där all producerad kod går igenom för att identifiera och dokumentera användandet av specifika moderna C++-metoder. Konkret undersöks huruvida felhantering implementerats med explicita returtyper snarare än undantag, om meddelanden mellan delsystem representeras *typsäkert*, och om buffertar hanteras utan manuell minneshantering. Utöver den manuella kodgranskningen användes systematiska kodgranskningar i samband med *Pull Requests* för att verifiera att idiomatisk C++23 tillämpades konsekvent.



5 Resultat

I detta kapitel redovisas projektets resultat, strukturerat utifrån de frågeställningar som presenterades i avsnitt 1.3.

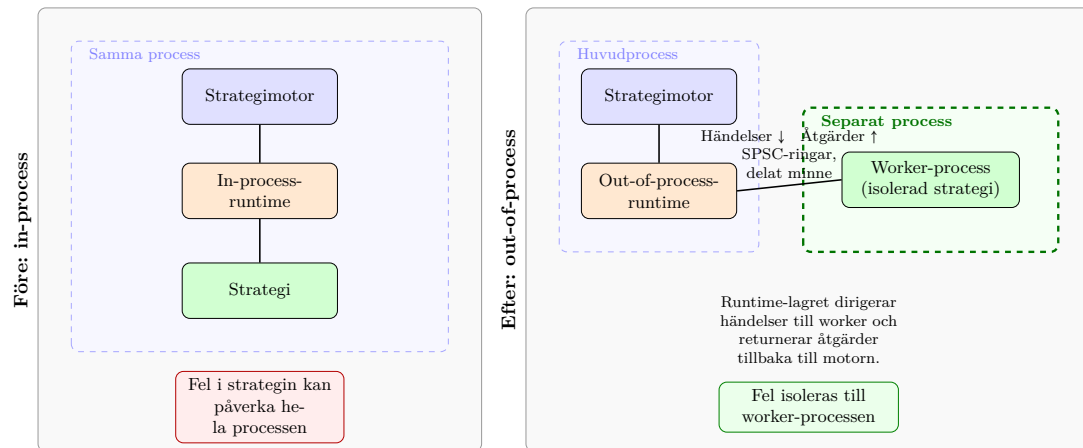
5.1 Hur en processisolerad arkitektur skapar värde för kunden

Ett av de primära resultaten är att strategikörningen inte längre behöver vara knuten till handelsmotorns kärnprocess. I den slutgiltiga arkitekturen kan strategier köras antingen in-process eller out-of-process, där det senare placerar strategin i en separat worker-process. Detta löser den huvudsakliga risk projektet syftade till att åtgärda: att ett fel i en enskild strategi kan leda till att hela motorn kraschar.

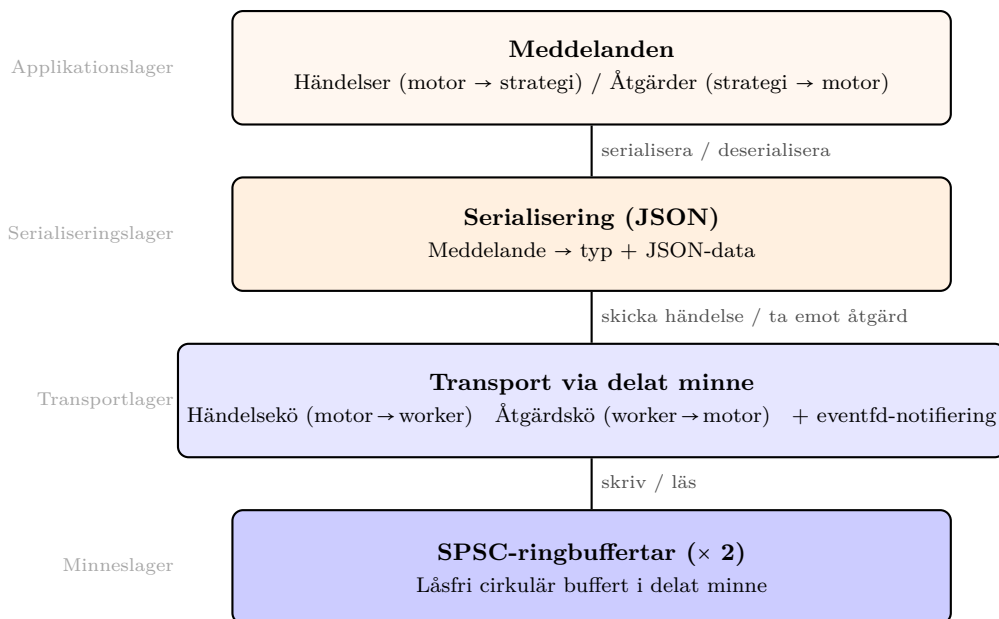
Den arkitektoniska skillnaden mellan den tidigare och den nya lösningen visas i Figur 5.1. Figuren gör tydligt att processisoleringen inte bara innebär att lägga en strategi i en ny process, utan även införandet av ett särskilt runtime-lager som hanterar kommunikation, väckningar och övervakning mellan motorn och workern.

Denna separation åstadkoms tekniskt genom ett mellanliggande runtime-lager som hanterar all kommunikation mellan strategimotorn och den isolerade worker-processen. Mellan dessa två systemdelar implementerades ett IPC-lager baserat på delat minne, bestående av två *SPSC*-ringbuffertar för dubbelriktad kommunikation. En ring transporterar händelser från motorn till workern, medan den andra transporterar åtgärder i motsatt riktning. För att undvika kostsam *busy-polling* kompletterades buffertarna med *eventfd*-baserade väckningar. Detta bevarade arkitekturens händelsestyrda natur och gjorde den kompatibel med motorns befintliga händelsehantering, vilket strukturellt visas i Figur 5.2.

Gruppen implementerade även en övervakningsmekanism för worker-processer. Genom att använda processövervakningsmekanismer i Linux kan motorn nu upptäcka om en worker avslutas, kraschar eller hänger sig. Vid ett sådant fel påbörjas en omstart med en exponentiell *backoff-fördröjning*. I samband med omstarten återupprättas IPC-kanalen, varpå motorn skickar över senast kända tillstånd gällande konto, inloggning, plånbok och positioner. Processisolering-

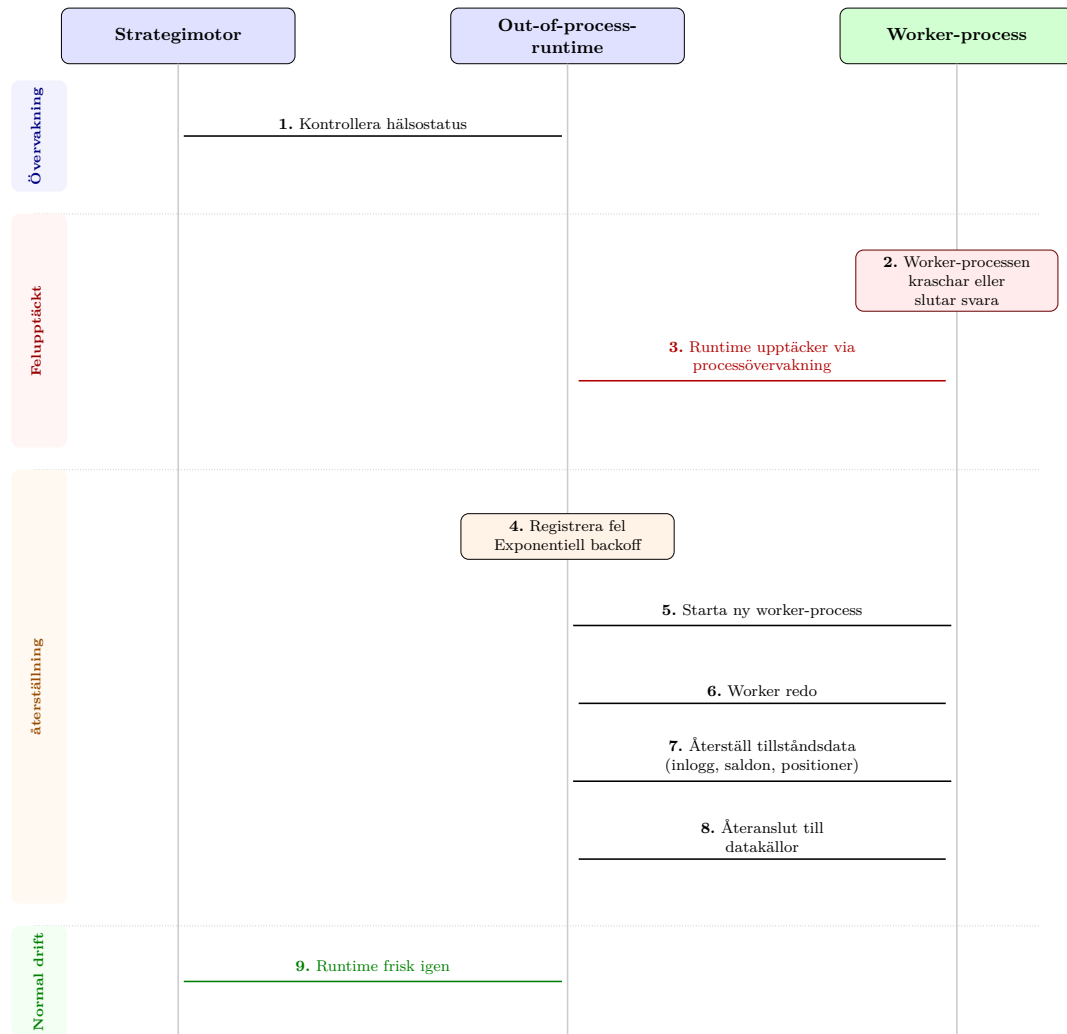


Figur 5.1: Jämförelse mellan in-process- och out-of-process-exekvering.



Varje meddelande passerar alla lager innan det når motparten. Samma stack används i båda riktningarna; rollen (motor eller worker) avgör vilken kö som används.

Figur 5.2: Transportstacken mellan strategimotor och worker-process.



Figur 5.3: Sekvensdiagram för felupptäckt och återhämtning.

en utökas därmed med en grundläggande återhämtningsförmåga, vilket direkt ökar systemets operationella tillförlitlighet för kunden.

Själva återhämtningskedjan illustreras i Figur 5.3. Där framgår hur runtime-lagret först upptäcker worker-fel, därefter startar om processen och slutligen spelar upp det senaste cachade tillståndet innan strategin återgår till normal drift.

Som ett komplement till arkitekturen togs även tester och benchmarkverktyg fram för att verifiera prestanda och robusthet. Dessa verktyg ger kunden möjlighet att rent kvantitativt mäta den latenskostnad som isoleringen innebär, och ställa denna mot vinsten i feltolerans.

5.1.1 Mätresultat

Analys av historisk marknadsdata visar att meddelandestorlekarna varierar kraftigt beroende på typ: frekventa statusuppdateringar är ca 130–150 bytes, orderboksuppdateringar har en median kring 410 bytes och initiala ögonblicksbilder kan nå hundratals kilobyte. Benchmarken nedan använde ett JSON-meddelande om 481 bytes, vilket är representativt för en typisk orderboksuppdatering. Serialiseringskostnaden skapas med meddelandets storlek; resultaten gäller därför primärt för meddelanden i denna storleksordning.

När programmet körs i in-process läge tar end-to-end flödet ca $46\mu s$, där deserialisering från JSON står för den enskilt största interna tidsåtgången med ca $22\mu s$.

Tabell 5.1: Latensjämförelse mellan in- och out-of-process (μs)

Läge	Payload (B)	p50 (μs)	p90 (μs)	p99 (μs)	p99.9 (μs)	Max (μs)
In-process	481	0.69	0.74	0.85	2.12	76.29
Out-of-process	481	36.33	51.07	119.70	424.17	630.75

Återhämtningstid efter strategikrasch testades också; i vår kravspecifikation gick gränsen för godkänt vid $<30s$, där väl godkänd var $<5s$ och utmärkt ansågs vara under en sekund. Uppmätt återhämtningstid var ca 118ms, vilket gör att vi klarar utmärktgränsen med god marginal. Återhämtningen består av två faser, en avsiktlig fördröjning på 100ms, och sedan en uppstartstid på ca 18ms.

5.2 Processbeskrivning

I projektet har gruppen arbetat med ett agilt arbetssätt med veckosprinter där varje vi under varje vecka planeringsmöte på måndagar beslutar hur projektets nästkommande veckas arbete planeras upp. Under ett möte på onsdagen varje vecka skedde även kvalitetsutvärdering av den nuvarande processen så att arbetsprocessen kontinuerligt uppdaterades.

5.2.1 Scrum och Kanban i praktiken

Från början var det tänkt att gruppen skulle applicera *parprogrammering*, detta visade sig vara ett ineffektivt arbetssätt då det inte fanns uppgifter nog för alla par då projektet hade för få delar för att kunna delas upp på ett effektivt sätt mellan alla par. Projektets höga tekniska svårighetsgrad gjorde även många uppgifter för svåra att tilldela enbart två gruppmedlemmar.

Istället applicerades mobbprogrammering med tre arbetsgrupper om tre personer. Gruppen lyckades efter en tids insättning lyckas identifiera tre övergripande huvudområden som tilldelades till varje arbetsgrupp. dessa huvudområden beskrivs i avsnitt 5.2.2 Huvudsakliga issues.

5.2.2 Huvudsakliga issues

- **Issue 4, Lokal spegling av marknadsdata:** Denna delgrupp ansvarade för att lösa hur isolerade strategiprocesser hanterar och läser av marknadsdata. Uppgiften innebar att designa en *mirrored cache* i arbetsprocesserna, vilket skulle tillåta strategierna att läsa av den absolut senaste marknadsinformationen utan att behöva göra tidskrävande nätverks- eller *RPC* anrop tillbaka till huvudmotorn. Ett centralt krav för detta arbete var att strikt kontrollera uppdateringsordningen. Den mirrored cache var tvungen att uppdateras före strategiernas *callbacks* aktiverades, för att garantera datakonsistens och bibehålla låg latens i systemets *hot path*.
- **Issue 5, Abstraktion av exekveringsmiljö:** Denna delgrupp fick i uppdrag att säkerställa att befintliga handelsstrategier kunde exekveras både in-process och isolerat out-of-process utan att kräva några anpassningar av strategiernas källkod. Arbetet innebar att dölja exekveringsmodellen från strategilagret genom att standardisera *instantiering*, *gränssnitt* och *callbacks*. Ett centralt krav för detta ansvarsområde var att valet av exekveringsläge uteslutande skulle kunna styras genom konfiguration, vilket krävde en enhetlig design av de underliggande transport- och tjänstelagren.
- **Issue 6: Telemetry och data:** Denna delgrupp ansvarade för att anpassa systemets insamling av data att skicka till centralservern, i den nya isolerade processarkitekturen.

Uppgiften innebar att utforma en lösning där strategiprocesser kunde kommunicera data direkt till en separat process via delat minne, vars uppgift är att skicka vidare datan till centralservern. Ett genomgripande krav för detta arbetsområde var att bibehålla det befintliga API:et oförändrat för strategierna, samt att säkerställa att *hotpath* inte belastades av detta dataflöde.

Gruppen var enad om att använda *Scrum* som beskrivet i avsnitt 3.2.2.

5.2.3 Utvärdering av processer

För att utvärdera processer hölls det veckovisa utvärderingsmöten där gruppen kunde diskutera de processer som applicerades och göra ändringar vid behov. Inför varje utvärderingsmöte skickades det ut en enkät. I enkäten kunde gruppmedlemmarna ranka olika aspekter av hur arbetet med sprintar har gått och lämna kommentarer och lyfta övriga önskemål. Dessa sammanfattades sedan av kvalitetssamordnaren och presenterades för gruppen; Därefter kunde gruppen diskutera och ta gemensamma beslut om processer behöver tas bort, läggas till eller ändras.

5.2.4 Huvudprocess

I detta avsnitt beskrivs hur gruppen valde att implementera *Scrum* och hur det användes inom projektet. Sprintarnas längd varierade men uppgick vanligtvis till två till tre veckor. Den längre sprintlängden gav gruppmedlemmarna tillräckligt med tid för att både fördjupa sin tekniska kunskap och slutföra tilldelade issues. Längre intervall mellan möten frigjorde mer tid för utvecklingsarbete och utbildning. Den ökade tiden mellan utvärderingsmötena bidrog även till mer innehållsrika diskussioner, då gruppmedlemmarna hunnit bilda välgrundade uppfattningar om arbetsprocessen.

5.2.4.1 Planering av sprints

Inför varje ny sprint så gick gruppen igenom kanbanbrädet och skapade nya issues. Gruppmedlemmar tilldelades därefter issues som de skulle jobba med. Antalet medlemmar som tilldelades berodde på vilket typ av issue det var. Utveckling gjordes i grupper av 3 och för rapportskrivning hade medlemmar möjligheten att skriva upp sig själva på ett issue vilket fungerade bra. I kanbanbrädet markerades det på varje issue om den var startad och vilka gruppmedlemmar som var tilldelade.

5.2.4.2 Planering av gemensamt arbete

Gruppen hade ett system där fyra gruppmedlemmar löpande bokade salar på förmiddag och eftermiddag. Detta fungerade väldigt bra och gruppen hade alltid någonstans att sitta och jobba tillsammans. Eftersom det alltid fanns salar bokade under arbetstimmar så hade alla gruppmedlemmar möjligheten att nå målet för antalet timmar utan att behöva sitta och arbeta ensamma. Att sitta tillsammans inom gruppen ökade även gruppens produktivitet jämfört när gruppmedlemmarna jobbade från distans.

5.2.4.3 Arbetsgång under sprint

Varje arbetsgrupp tilldelad en issue ansvarade för att jobba på den tillsammans. Varje arbetsgrupp hade egna kommunikationskanaler i *Discord* där de kan diskutera och skicka relevant information för deras issue. Utvecklingen för varje issue gjordes på separata *branches* i *GitLab*, detta gjorde det möjligt för arbetsgrupperna att skriva kod utan att påverka den huvudsakliga koden. När en arbetsgrupp anser sig vara klara med sin issue så kan de skapa en *merge-request* för att lägga in den nya koden på projektets huvudsakliga *branch*. För att en merge-request

ska gå igenom behöver den granskas, testas och godkännas av två gruppmedlemmar utanför arbetsgruppen. Därefter blir arbetsgruppen tilldelade ett nytt issue av utvecklingsledaren.

5.2.4.4 Avslutande av sprint

Vid avslutandet av en sprint hölls en sprint review och en sprint retrospective under ett längre möte, på mötet gjorde gruppen det som beskrivs i avsnitt 5.2.3 och 5.2.4.1. Under detta möte förväntades alla gruppmedlemmar närvara. Först utvärderades den föregående sprinten och de processer som har använts, därefter utvärderades resultaten som åstadkommit och vilka uppgifter som avklarats. Till sist går gruppen tillsammans igenom projektets kanbanbräde för att ta bort, skapa nya eller uppdatera nuvarande uppgifter och issues och hur dessa ska fördelas.

5.2.5 Förändring av huvudprocess

En förändring inom arbetets processer som gjordes var att issues och uppgifter var tänkta att göras med parprogrammering alternativt ensam om det var en mindre uppgift. Gruppen insåg via utvärderingsmöten under arbetets gång att projektet inte passade att genomföras på detta sättet. Anledningen varför var att uppgiften först och främst enbart berörde en del av systemet vilket gjorde att det var svårt att dela upp den i flera delar utan att ha för mycket beroenden mellan delarna. Dessutom är projektet tekniskt tungt jämfört med gruppens tekniska kunskap.

Detta ledde till att gruppen istället bestämde sig för att dela upp sig i större grupper och använda sig av mobbprogrammering, då gruppen märkte att den begränsande faktorn i projektet var en hög svårighetsgrad på den nuvarande koden. Detta gjorde att det var nyttigt att låta flera gruppmedlemmar göra forskning inom samma problem för att förhindra att en gruppmedlem lägger för mycket tid på att läsa befintlig kod. När mobbprogrammering applicerades kunde tre till fyra gruppmedlemmar sitta tillsammans och forska på lösningar till uppgiften som skulle lösas. Då mängden kod som skulle skrivas även var relativt liten jämfört med liknande projekt fungerade det även bra att skriva kod i grupper av tre till fyra personer. Eftersom koden som skrevs var relativt komplicerad jämfört med gruppmedlemmarnas tekniska kunskap så var det även positivt att flera gruppmedlemmar hängde med i utvecklingen och kunde förstå den koden som skrevs jämfört med om det bara hade varit en person som programmerade, detta underlättade vidare arbete och gav gruppen en bättre allmän förståelse över hela projektet.

Denna processförändring påverkade även andra delar av projektets processer, bland annat så ökades längden per sprint för att varje grupp skulle ha tillräckligt med tid för sitt tilldelade issue, denna avvägning var värd att göra då det resulterade i en högre kvalitet på koden som skrevs och även förhindrade att gruppmedlemmar körde fast i arbetet.

5.3 Erfarenheter från projektets genomförande

Projektets praktiska genomförande kom att delas in i tre distinkta faser. Arbetsgången inleddes med ett teoretiskt fokus på utbildning och kodförståelse av kundens plattform. Detta visade sig dock ineffektivt för att faktiskt komma igång med implementationen, vilket ledde gruppen in i nästa fas: försök med mobbprogrammering i större grupper. Inte heller detta fungerade tillfredsställande på grund av otydlig ansvarsfördelning och lågt tempo.

Vändpunkten nåddes i den tredje fasen under övergången till ett arbetsflöde drivet av agentiskt ingenjörskap. Agenterna (huvudsakligen via *OpenAI-verktyg*) matades med systemets kontext för att iterativt och snabbt ta fram grova arkitektoniska utkast. Med denna AI-genererade grund på plats formulerade gruppen utvalda Git-issues (redovisade i metodkapitlet) och delade in sig i dedikerade delgrupper kring dessa problem. Övergången till drastiskt mindre

ansvarsområden parat med en agent-genererad basarkitektur var *de facto* avgörande för produktiviteten. Det var detta metodskifte som tröskade bort mycket initial tyngd och i slutändan möjliggjorde att ett fungerande, isolerat multiprocesssystem i C++²³ kunde levereras inom projektets hårda ram om 400 timmar per person.



6 Diskussion

I detta kapitel förs en diskussion kring projektets genomförande, specifikt vad som har fungerat väl, de utmaningar gruppen har stött på samt de lärdomar som kan vara av intresse för framtida utvecklingsarbeten. Vidare diskuteras de resultat som presenterats i föregående kapitel 5.

6.1 Hur skapar arkitekturen värde för kunden

Det tydligaste kundvärdet återfinns i hur robusthet nu hanteras som en fundamental egenskap i arkitekturen, snarare än som ett beroende av felfri strategikod. Att isolera strategier i separata processer neutraliserar den största enskilda driftolyckan: att en felande handelsalgoritm orsakar totalstopp i hela plattformen. I en HFT-miljö, där ett avbrott bär med sig snabba konsekvenser både ekonomiskt och i form av marknadsförtroende, är denna isolation värdefull.

Å andra sidan medför lösningen en oundviklig ökning av systemkomplexiteten. Processisolering introducerar IPC-köer, processövervakning, automatiserad omstart och komplicerad hantering av återställningstillstånd. Detta i sin tur genererar fler potentiella felkällor och ställer högre krav på testning jämfört med in-process-körning. Projektets slutsats är därför att processisolering har ett underhållspris, men att det numera går att tillämpa effektivt med bibehållen prestandakontroll.

6.2 Erfarenheter givna av projektet

Projektets erfarenheter understryker att ett tekniskt avancerat utvecklingsarbete sällan fallerar enbart på implementationen, utan oftare på svårigheter i att etablera en delad systemförståelse i gruppen. Att gruppen initialt kämpade med att omsätta teoretisk C++-utbildning till konkret programmering belyser ett väsentligt metodproblem: om utbildningen inte existerar i direkt anslutning till den kunds specifika kodbasen är det praktiska värdet begränsat.

När arbetsgruppen senare delades in i tre fristående arbetsgrupperingar visade det sig dock vara en effektivare metod. Formatet tvingade fram en skärpt ansvarsfördelning och tillät en djupdykning i systemets avgränsade delar som inte var uppnåelig i det större formatet. Nackdelen med denna uppstyckning var en påtaglig *kunskapsisolering* inom gruppen, där få fullt

ut greppade systemet som helhet på en djup nivå. Följaktligen fungerade dokumentation, diskussioner över systemanatomin och kodgenomgångar som utmärkta verktyg för att motverka total *kunskapsisolering*.

6.2.1 Utbildning och det tidiga skedet

Att fördela arbetet effektivt i en projektgrupp på nio personer visade sig inledningsvis vara en stor utmaning. En primär orsak till detta var en initial brist på djupgående förståelse för den befintliga kodbasen, specifikt gällande dataflöden och hantering av in- och utdata. Trots att en betydande del av projektets inledande fas hade allokerats till utbildning inom C++, resulterade detta inte i den praktiska förståelse som krävdes för att påbörja implementationen. I efterhand kan gruppen konstatera att utbildningen borde ha fokuserat i mycket högre utsträckning på kundens specifika kodbas snarare än generella språkkoncept.

På grund av kodbasens komplexitet och den branta inlärningskurvan visade det sig svårt för gruppen att initialt komma igång. För att komma över denna tröskel och påskynda implementering användes diverse AI-verktyg som bollplank men även för kodgenerering (huvudsakligen OpenAI Codex/ChatGPT) för att bygga och forma projektets arkitektoniska grundpelare. När de grundläggande strukturerna fanns på plats kunde gruppen utgå från dessa samt ansvara för att förfina, integrera och kvalitetssäkra koden samt fatta nödvändiga arkitekturbeslut under den fortsatta itereringen.

6.3 Systemanatomi som stöd

Projektets resultat bekräftar att systemanatomin fyllde en viktig funktion, men främst som ett kommunikationsverktyg och inte som en strikt teknisk ritning. Istället för att följas mekaniskt användes diagrammen som ett visuellt stöd för att gruppen enklare skulle kunna diskutera gränssnitt och ansvarsområden mellan de tre delgrupperna. I ett projekt där en studentgrupp utvecklar en isolerad del av ett annars ofantligt mycket större system, var denna typ av gemensam överblick högst relevant. Skapandet och underhållandet av en systemanatomi underlättade också mycket i arbetet med agentisk ingenjörskap. Systemanatomin gav en tydlig bild av hur projektgruppen ville att koden skulle fungera fungera när den var färdig. Detta gav större möjligheter att guida agenterna med precision.

Den största utmaningen med en systemanatomi är underhållet. När gruppens kod förändrades snabbt visade det sig viktigt att dokumentation och diagram omedelbart uppdaterades för att stämma överens med verkligheten. Under projektets avslutande sprinter tappades denna rutin delvis, vilket snabbt ledde till att anatomin förlorade sitt värde och slutade användas som referens. Slutsatsen är att en systemanatomi enbart bär sitt värde in i integrationsfasen om projektgruppen har disciplin nog att kontinuerligt hålla den uppdaterad.

6.4 Modern C++ roll i projektet

Resultaten visar att modern C++ inte var den avgörande faktorn för arkitekturomställningen. Den nya arkitekturen bygger i grunden på operativsystemets funktionalitet, primärt Linux-processer och delat minne. Språket åstadkom alltså inte processisoleringen i sig, utan fungerade endast som det verktyg gruppen använde för att bygga systemet ovanpå dessa OS-funktioner.

Givet detta fyller den nya standarden en viktig roll för systemets robusthet på kodnivå. Genom explicit felhantering och typsäkra meddelanden minskade komplexiteten avsevärt i implementationen. Arkitekturen hade visserligen gått att bygga med äldre C++-standarder, men funktionerna i C++23 gjorde koden betydligt renare och framförallt enklare för kunden att underhålla på sikt.

6.5 Agentiskt ingenjörskap

Med tanke på att den nya arkitekturen i hög grad är framtagen med stöd av agentiskt ingenjörskap och AI, uppstår oundvikligen akademiska frågor kring verktygens påverkan på reell inlärninng och systemets slutgiltiga tillförlitlighet.

En initial farhåga är ifall nyttjandet av AI hämmade inlärandet av *C++*. Projektets erfarenheter pekar snarare på att metodiken accelererade inläringen av projektspecifik kunskap, även om denna observation bygger på gruppens subjektiva upplevelse och inte på en kontrollerad jämförelse med ett alternativt tillvägagångssätt. Gruppen undgick att fastna i trivial kod och växlade omedelbart upp fokus mot *top-down* systemarkitektur, IPC-mekanismer och prestandaoptimering.

I den initiala utbildningsfasen studerades *C++ bottom-up* på ett utforskande sätt. Detta gjorde att medlemmarna studerade saker som i slutändan inte var användbara i just detta projekt, men som ändå ökade förståelsen kring *C++* på ett bredare plan. När man studerar något utforskande ställer man sig ofta frågan ”kan detta fungera i kontexten av vårt projekt?”, men när man studerar *top-down* blir frågan istället ”varför fungerar detta i vår kontext?”. En annan aspekt man bör ta hänsyn till är hur mycket motivation spelar in vid inlärninng. När materialet man studerar är starkt kopplat till något man vet är relevant, kan det bli lättare att minnas varför det är viktigt och hur det fungerar.

Arbetet blev besläktat med hur professionell *kodgranskning* utförs; gruppmedlemmarna var tvungna att förstå den tillgodogjorda koden exakt lika bra som om de författat den själva, annars hade det varit svårare att hitta prestandaförbättringar och garantera isolering. Detta system skapade dock en inneboende utmaning: Det är ofta tilltalande att ta genvägar och okritiskt acceptera genererad kod man inte fullt ut analyserat. Agentiskt ingenjörskap ställer därmed paradoxalt nog *högre* krav på ingenjörens disciplin än vad traditionellt ingenjörskap gör.

Slutligen infinner sig frågan: är det ansvarsfullt att lita på AI-genererad kod i ett HFT-system där varje enskild mikrosekund av latens värderas högt, och utrymmet för fel är noll? Projektets erfarenheter tyder på att det kan vara försvarbart, men enbart under starka förutsättningar. Det är kritiskt att särskilja metodiken, så som den teoretiskt definieras [11], från så kallad *vibe coding*. All genererad kod, oavsett om den berörde latenskänsliga vägar eller mer systemdelar i periferin, genomgick samma granskningsprocess. Varje genererat segment testades mot minnesverktyg, tester och isolerade mock-adaptrar innan det integrerades. Säkerhetsgarantin kom alltså inte från AI-modellerna i sig, utan från det skyddsnät av tester och iterativa granskningar som utvecklingsprocessen bestod av.

En risk med att använda sig av AI-assistenter är att man kan generera mycket kod snabbt [28], men att den sedan kräver mer tid att debugga [15], vilket kan påverka hur svår den är att underhålla då den som sitter med koden eventuellt inte känner till den på samma djup som någon som skrivit varje rad. Detta hade varit ett problem för kunden oavsett om AI-verktyg använts, då kundens personal inte har varit delaktiga i denna process. Där blir det istället viktigare att koden är väldokumenterad så att de som arbetar vidare på den har lättare att sätta sig in i den.

6.5.1 Varför en mänsklig grupp behövdes

En relevant fråga är varför en studentgrupp överhuvudtaget behövdes om AI-verktyg kan generera kod. Svaret är att AI-agenter inte kan släppas lösa på en komplex kodbas utan mänsklig styrning. Att använda agentiskt ingenjörskap kräver extrem noggrannhet i att läsa, förstå och validera den kod som produceras. Agenten saknar domänspecifik förståelse: den väljer

konsekvent generiska standardbiblioteksstrukturer framför de optimerade, datastrukturer som är specialanpassade för kodbasen och som dessutom redan finns i kodbasen och som är kritiska för att uppnå latenskraven. Gruppens roll var att tillhandahålla denna domänkontext, ställa rätt frågor och fatta de arkitekturbeslut som agenten inte hade underlag för. Det bör också noteras att kunden själv aktivt uppmuntrade användningen av AI-verktyg i projektet, med insikten att dessa verktyg idag är en förutsättning för att vara konkurrenskraftig inom mjukvaruutveckling.

6.5.2 Långsiktig hållbarhet av AI-assisterad utveckling

Frågan om huruvida AI-assisterad utveckling är hållbar på lång sikt är komplex. Å ena sidan finns en risk att AI-genererad kod ackumulerar teknisk skuld snabbare än manuellt skriven kod, särskilt om framtida utvecklare inte fullt ut förstår de lösningar som producerats. Å andra sidan förbättras AI-modellerna kontinuerligt, vilket innebär att de brister som finns idag sannolikt minskar över tid. En viktig observation från projektet är att agentiskt ingenjörskap ställer höga krav på dokumentation och kodstruktur. Eftersom varje ny AI-session startar utan tidigare kontext måste kodbasen och dess dokumentation vara tillräckligt tydlig för att en agent ska kunna producera relevanta resultat. Denna egenskap kan paradoxalt nog leda till bättre underhållbarhet, förutsatt att projektgruppen disciplinerat upprätthåller dokumentationskvaliteten.

6.5.3 Begränsningar i testningens täckning

Avslutningsvis bör det erkännas att projektets testsvit primärt verifierar funktionalitet och prestanda i kontrollerade scenarier. Det finns en risk att AI-genererad kod har introducerat subtila fel eller oväntade beteenden som de befintliga testerna inte täcker. Denna risk förstärks av att gruppen saknade den djupa expertkunskap som krävs för att identifiera alla potentiella felkällor i en HFT-miljö. Den testning som genomförts kan därför inte ses som en fullständig garanti för korrekthet, utan snarare som en rimlig verifiering givet projektets resurser och tidsramar.

6.6 Metod

Den mest grundläggande begränsningen i arbetet är gruppens avsaknad av djup expertkunskap inom HFT-utveckling och systemprogrammering på den nivå som projektet kräver. Även om utbildningsinsatser och AI-verktyg delvis kompenserade för detta, innebär bristen att gruppen kan ha missat känsliga prestandaproblem eller designfel som en erfaren systemutvecklare hade upptäckt. Denna begränsning genomsyrar hela arbetet och bör beaktas vid tolkning av samtliga resultat.

Utöver detta har all verifiering och testning skett med simulerad marknadsdata via mock-adaptrar, istället för system uppkopplade mot verkliga börser. Resultaten visar att arkitekturen kring processisolering och IPC fungerar som förväntat i en kontrollerad miljö, men det saknas mätdata från produktion gällande hur väl kommunikationskanalerna klarar extrem nätverkstrafik vid kraftiga marknadsrörelser. För att garantera stabiliteten i skarpt läge behöver systemet testas med verklig marknadsdata.

I anslutning till denna kontrollerade testmiljö bör också tekniska-felkällor i mätdata beaktas. I den ärvda end-to-end latenstestaren exekveras flödet med minimalistisk payload, då den serialiserade JSON-filen enbart blir 111 bytes. Då en serialiserad handelsorder blir minst 481 bytes, innebär det att de end-to-end tider som utvinns är mindre än de kan förväntas vara i drift, där marknadsdata-filer från externa API:t kan vara många gånger större.

Både end-to-end och prestandatesterna för IPC lider av samma strukturella brist: dataläsning sker från en identisk minnesplats under alla iterationer. Detta leder till att cache-missar undviks helt. Alla tester körs också i en annan miljö än den unika miljö som programvaran kommer att användas i, och processorns prestanda kommer att påverka prestandan på dessa.

Prestandatesterna kördes i en virtualiserad miljö, vilket introducerade en osäkerhet i form av hypervisor-schemaläggning och resurskonkurrens med värdsystemet. Detta kan ha påverkat de extrema max-värden som registrerades under latens-testningen.

Alternativet till mock-adaptrar hade varit att bygga en dedikerad *backtester* för att spela upp historisk börsdata från plattformens dataleverantör. Leverantören tillhandahåller historisk data i komprimerat format som är direkt kompatibelt med systemets befintliga datastrukturer. Den tekniska utmaningen ligger i att koppla ur den levande nätverksinfrastrukturen; plattformens hanterare för marknadsdata behöver kompletteras med en avläsare för komprimerade dataströmmar, och orderläggningen måste ersättas med marknadssimulerande logik. Givet tidsramen bedömdes utvecklingen av denna simuleringsarkitektur falla utanför projektets ramar, medan skapandet av en robust miljö för historisk testning kvarstår som ett övergripande åtagande.

Samtidigt är det just fokuset på den specifika koden som ger rapporten sitt värde. Arbetet har handlat om att lösa ett tekniskt problem i en befintlig HFT-motor, snarare än att teoretisera över generell IPC-prestanda. Att lösningen och testerna är helt skraddarsydda efter kundens kodbas är därför inget problem i detta sammanhang, utan en förutsättning för att arbetet alls skulle kunna genomföras.

6.7 Arbetet i ett vidare sammanhang

Arbetet med processisolering i en HFT-motor bör inte enbart betraktas som ett tekniskt arkitekturproblem, då systemet används i en finansiell kontext, tekniska beslut kan få konsekvenser som sträcker sig utanför själva programvaran. Ett fel i en handelsstrategi eller en misslyckad återhämtning kan leda till ekonomiska förluster, driftstörningar och minskat förtroende för systemet.

Genom att isolera strategier i separata processer begränsas effekten av minnesfel eller krascher i en enskild strategi. Detta kan ses som positivt ur ett säkerhetsperspektiv eftersom systemet får bättre förutsättningar att fortsätta fungera trots fel i enskilda komponenter. Samtidigt innebär lösningen inte att systemet blir riskfritt då systemet fortfarande kan krascha om IPC-lagret, *supervisor*-logiken eller återstartsmekanismen brister. Även den nya arkitekturen orsaka oönskade effekter. Därför måste lösningen ses som en riskreducerande åtgärd snarare än en fullständig garanti för säker drift.

Det finns även en etisk dimension i att utveckla teknik för högfrekvenshandel. HFT-system kan bidra till ökad likviditet och effektivare handel, men de kan också förstärka problem som marknadsinstabilitet, teknisk kapprustning och ojämlik tillgång till marknadsinfrastruktur. Det har även hänt att HFT-system används för marknadsmanipulation [29]

Projektet har inte innefattat utveckling av handelsstrategier eller beslutslogik för köp och försäljning. Dock förbättrar arbetet den tekniska infrastrukturen som möjliggör sådan handel, därför är det relevant att reflektera över att även infrastrukturförbättringar kan bidra till större samhällseliga effekter. Ett mer robust system är positivt för kunden, men kan samtidigt ses som en del av en bransch där teknisk överlägsenhet ger konkurrensfördelar som inte alla marknadsaktörer har möjlighet att uppnå.

Ur ett ekonomiskt perspektiv skapar arbetet värde för kunden genom att minska risken för driftavbrott och göra systemet mer motståndskraftigt mot fel i enskilda strategier. Detta är särskilt viktigt i miljöer där mycket små tidsfördröjningar eller korta avbrott kan få stora konsekvenser. Samtidigt innebär den nya arkitekturen ökad komplexitet. Fler processer, delat minne, övervakningslogik och återhämtningsmekanismer gör systemet svårare att förstå, testa och underhålla. Den ekonomiska nyttan måste därför vägas mot den långsiktiga kostnaden på grund av ökad teknisk komplexitet.

Även hållbarhetsaspekter kan diskuteras. HFT-system ställer höga krav på beräkningskraft, vilket kan leda till ökad resursförbrukning. Projektets fokus har varit att förbättra robusthet och feltolerans, inte att minska resursförbrukningen. Samtidigt kan ett stabilare system indirekt minska resursförbrukning kopplad till krascher följt av omstart. Denna effekt är dock begränsad och bör inte överdrivas.

Projektets användning av AI-baserade verktyg väcker också frågor om ansvar och kunskapskontroll. Generativ AI kan vara ett effektivt stöd för att förstå stora kodbaser och ge förslag på arkitektur men det innebär samtidigt risker. Kod som genereras av AI kan vara felaktig, svår att verifiera eller bygga på antaganden som inte stämmer med övrig kod. I ett system med höga krav på korrekthet kan utvecklare därför inte överlåta ansvar till verktyget. Den mänskliga rollen blir snarare att granska, testa, förstå och motivera de lösningar som används. AI-användningen kan därmed vara värdefull, men endast om den kombineras med tydliga granskningsprocesser och tekniskt ansvarstagande.

Sammanfattningsvis visar projektet att processisolering kan vara ett viktigt steg mot mer robusta HFT-system, men också att tekniska förbättringar alltid bör analyseras i sitt större sammanhang. Lösningen minskar vissa risker, men introducerar samtidigt nya former av komplexitet och nya krav på testning, övervakning och underhåll. Arbetets bredare värde ligger därför inte enbart i den färdiga implementationen, utan även i insikten att låglatenssystem måste utvecklas med medvetenhet om ekonomi, etik, hållbarhet och ingenjörsmässigt ansvar.



7 Slutsats

I detta avslutande kapitel sammanfattas projektets slutsatser i relation till de initiala frågeställningarna. Vidare diskuteras arbetets konsekvenser för målgruppen, följt av konkreta rekommendationer för framtida utveckling.

7.1 Besvarande av frågeställningar

Denna sektion drar slutsatser om de frågeställningar som rapporten inleds med i avsnitt 1.3.

7.1.1 Värde för kunden

Projektet kan i första hand konstatera att en processisolerad arkitektur skapar ett direkt och påtagligt kundvärde genom att eliminera plattformens största operationella svaghet. Genom att exekvera strategierna i ett isolerat Untrusted-läge säkerställs att fatala minnesfel och krascher inte tillåts påverka den betrodda handelsmotorn. Den implementerade IPC-lösningen, baserad på låsfria SPSC-ringbuffertar i kombination med eventfd-signaler och delat minne, bevisar att denna feltolerans kan uppnås med bibehållen händelsestyrd karaktär. Även om arkitekturen medför en ökad systemkomplexitet, innebär införandet av en autonom övervakningsmekanism att systemet nu kan återhämta sig på egen hand via en kontrollerad omstartsprocess. Robusthet är därmed inte längre ett tyst beroende av felfri strategikod, utan är fundamentalt inbyggt i plattformens infrastruktur.

7.1.2 Erfarenheter

Vad gäller erfarenheter från projektets genomförande är den tydligaste slutsatsen att allmän utbildning inom ett tekniskt område endast kan användas i begränsad utsträckning för att uppnå en kompetent teknisk kunskapsnivå. En mer fokuserad utbildning med fokus på kundens specifika kodbas och projektes arkitektur visade sig vara betydligt mer effektiv. Användningen av agentiskt ingenjörskap för att tidigt i utvecklingsprocessen iterativt ta fram arkitektoniska grunder var värdefullt och hjälpte till att bygga en grund arbetet kunde utvecklas ovanpå. Vidare är en viktig erfarenhet i gruppen att den optimala utvecklingsmetoden kan variera över tid inom ett projekt. När vi påbörjade gemensam programmering i en stor mobbgrupp

var detta en bra metod för att bygga den gemensamma kunskapsnivån. Stora mobbgrupper visade sig senare ineffektivt och arbetet accelererade i samband med en uppdelning i mindre grupper med fokus på ett issue vardera.

7.1.3 Stöd av systemanatomin

Den största nyttan med systemanatomin var att den klargjorde vad projektgruppen inte skulle röra. Genom att visuellt rita upp den ursprungliga kodarkitekturen och jämföra den med den önskade målbilden blev det uppenbart var i systemet arbetet skulle ske. Det gav gruppen trygghet i att veta vilka komponenter som behövde byggas om, medan merparten av plattformen kunde hanteras som en sluten svart låda.

Konkret innebar det att strategimotorn, *runtime*-abstraktionen, worker-processen och *IPC-lagret* kunde avskiljas som en isolerad komponentgrupp med tydliga in- och utgångar. Systemanatomin hjälpte därmed gruppen att utforma systemet på rätt abstraktionsnivå. Istället för att försöka överblicka handelsplattformen i sin helhet kunde fokus riktas mot ett avgränsat dataflöde: från motor till runtime, från runtime till worker, och tillbaka samma väg mot de befintliga order- och marknadsdatavägarna.

Dessutom fungerade systemanatomin som ett effektivt kommunikationsverktyg internt. När undergrupperna arbetade parallellt skapade arkitekturbilderna en gemensam referensram för integrationspunkterna, vilket säkerställde att design och slutgiltig testning stämde väl överens med de tidiga tekniska besluten.

7.1.4 Användning av modern C++ inom multitrådning och prestandaorienterad kod

Resultaten indikerar att moderna språkfunktioner från C++ underlättade omställningen mot en modulär arkitektur, men att standarden i sig inte utgör kärnan i lösningen. I den underliggande implementationen utnyttjades moderna C++-funktioner för explicit felhantering inom IPC-lagret, typsäkra meddelanden mellan motor och worker, minnessäker hantering av data i ringbuffertarna samt för att uttrycka frånvaro av data på ett sätt som kompilatorn kan verifiera.

Tillsammans bidrog dessa komponenter till tydligare gränssnitt mellan delsystemen. Runtime-abstraktionen, transportlagret och serialiseringslagret kunde definieras på ett sätt som gjorde den resulterande implementationen läsbar, testbar och typsäker. Det slutgiltiga resultatet är dock inte att modern C++ egenhändigt möjliggjorde processisoleringen. Stödet från standarden bidrog snarare till ett mer robust och lättunderhållet system, medan den faktiska arkitekturomställningen förlitade sig på operativsystemets processmodell och Linux-specifika systemanrop.

7.2 Utbildning för kandidatprojekt

I efterhand kan gruppen också dra slutsatsen att den inledande teoretiska utbildningen inte var tillräcklig för att skriva effektiv kod. Som motåtgärd konstruerades därför ett kompletterande internt utbildningsmaterial. Detta material hade en starkare förankring i den specifika kodbasen – exempelvis kring systemets centrala gränssnitt, händelsehantering, IPC och låsfria datastrukturer – vilket skapade ett mer enhetligt tekniskt vokabulär inom gruppen.

Projektet visade därtill på vikten av styrande dokument. Projektplan, kravspecifikation, testplan, kvalitetsplan och arkitekturdokument användes löpande som praktiska arbetsredskap, inte enbart som formella kravleveranser. Lärdomen är att dokumentation och kodimplemen-

tation behöver utvecklas parallellt för att en grupp på nio personer ständigt ska ha samma bild av vad som faktiskt ska byggas.

7.3 Konsekvenser för målgruppen

Resultaten från detta arbete bär direkta konsekvenser för plattformsutvecklare och systemarkitekter inom högfrekvenshandel.

Detta projekt demonstrerar i praktiken att ansvaret för stabiliteten kan frikopplas från strateginivån och flyttas uppåt till systeminfrastrukturen. Genom att tillhandahålla en arkitektur där strategier faktiskt "tillåts" krascha, kan utvecklingstakten för nya, komplexa algoritmer accelereras, eftersom de förödande konsekvenserna av ett feldrabbat minnesexperiment i produktion drastiskt sänks. Möjligheten att mätbart och per maskin konfigurera exekveringsläge ger målgruppen ett konkret verktyg för att kvantitativt väga stabilitet mot latenskrav för varje specifik strategi.

7.4 Framtida arbete

Med utgångspunkt i projektets resultat och identifierade begränsningar föreslås följande konkreta områden för det fortsatta utvecklingsarbetet hos Alexandria Invest:

Infasning av historisk marknadsdata från CryptoStruct

Eftersom all inledande testning av IPC-mekanismen och supervisor-logiken har genomförts med hjälp av relativt enkla mock-adaptrar, kvarstår ett tydligt behov av att testa systemet i en verklighetsnära miljö. Innan plattformen hanterar faktiskt kapital bör arkitekturen utvärderas mot en intern *backtester* kapabel att mata systemet med historiska ordersekvenser från plattformens dataleverantör, CryptoStruct. Särskilt brådskande är att utvärdera IPC-köernas hantering av systematiskt *backpressure* i historiskt baserade scenarier präglade av extrem marknadsvolatilitet.

Prestandaprofilering i produktionsmiljö

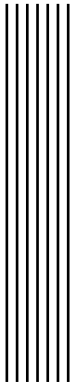
En omfattande kvantitativ benchmarkstudie måste utföras på målsystemets avsedda produktionshårdvara. Genom att rigid jämföra P99- och P999-latenserna, det vill säga den 99:e respektive 99,9:e percentilen av latensfördelningen, mellan Trusted- och Untrusted-lägen – implementerat med strikt *CPU-pinning* – kan en pålitlig baslinje fastställas. Detta är essentiellt för att framöver kunna bedöma vilket exekveringsläge handelslogik bör allokeras till.

Dataöverlevnad vid kraschåterhämtning

I dagsläget kräver en omstart av en Untrusted-strategi att motorn återsänder senast kända tillståndsdata såsom inlogg, saldon och aktiva positioner. En framtida utökning av uppdraget vore att utforska implementationen av en snabb, delad minnes-cache för detta strategitillstånd. Detta skulle säkerställa tillståndsoverlevnad även när operativsystemet frigör processminnet, under premissen att lösningen inte introducerar synkroniseringsoverhead i uppdateringsflödet (Hot Path).

Utökad övervakning av enskilda strategier

Ett naturligt nästa steg är att fördjupa övervakningen av varje isolerad strategi. Genom att löpande mäta minnesförbrukning och exekveringstid inifrån själva strategiprocessen ges plattformsoperatörer möjlighet att identifiera prestandaförsämringar och varningssignaler proaktivt, innan de leder till en regelrätt krasch.



Referenser

- [1] CryptoStruct GmbH. *CryptoStruct – Low latency crypto trading platform*. <https://www.cryptestruct.com/>. Hämtad: 2026-04-15.
- [2] Linköping University. *TDDE68 Concurrent Programming and Operating Systems – Syllabus*. Accessed: 2026-05-06. 2025. URL: <https://studieinfo.liu.se/en/kurs/TDDE68/vt-2025#syllabus>.
- [3] cppreference.com. *C++23*. <https://en.cppreference.com/w/cpp/23.html>. Hämtad: 2026-03-24.
- [4] Git SCM. *Git Documentation*. <https://git-scm.com/about>. Hämtad: 2026-03-24.
- [5] GitLab Inc. *GitLab features*. <https://about.gitlab.com/platform/>. Hämtad: 2026-04-11. 2026.
- [6] Jeremy Holcombe. *Vad är Docker: En komplett guide*. <https://kinsta.com/se/blog/vad-ar-docker/>. Hämtad: 2026-03-23. 2025.
- [7] scrum.org. *Scrum, What is Scrum?* <https://www.scrum.org/learning-series/what-is-scrum/>. Hämtad: 2026-03-24.
- [8] scrum.org. *Scrum, What is a produkt backlog?* <https://www.scrum.org/resources/what-is-a-product-backlog>. Hämtad: 2026-03-24.
- [9] scrum.org. *Scrum, The scrum team, 2.3 What is a product owner?* <https://www.scrum.org/learning-series/what-is-scrum/the-scrum-team/what-is-a-product-owner>. Hämtad: 2026-03-24.
- [10] scrum.org. *Scrum, The scrum team, 2.1 What is a scrum master?* <https://www.scrum.org/learning-series/what-is-scrum/the-scrum-team/what-is-a-scrum-master>. Hämtad: 2026-03-24.
- [11] OpenAI. *Harnessing Agentic Engineering*. <https://openai.com/sv-SE/index/harness-engineering/>. Hämtad: 2026-04-15.
- [12] Wikipedia contributors. *Vibe coding — Wikipedia, The Free Encyclopedia*. [Online; accessed 12-May-2026]. 2026. URL: https://en.wikipedia.org/wiki/Vibe_coding.
- [13] Elyson De La Cruz, Hanh Le, Karthik Meduri, Geeta Sandeep Nadella* och Hari Gonnaygunta. "Redefining the Programmer: Human-AI Collaborations, LLMs, and Security in MModern Software Engineering". I: *Tech Science Press* (2025).

- [14] Timothy Cook. "Adults Lose Skills to AI. Children Never Build Them". I: *Psychology Today* (mars 2026). URL: <https://www.psychologytoday.com/us/blog/the-algorithmic-mind/202603/adults-lose-skills-to-ai-children-never-build-them>.
- [15] Judy Hanwen Shen och Alex Tamkin. *How AI Impacts Skill Formation*. 2026. arXiv: 2601.20245 [cs.CY]. URL: <https://arxiv.org/abs/2601.20245>.
- [16] IBM. *Atomic operations*. <https://www.ibm.com/docs/en/aix/7.3.0?topic=services-atomic-operations>. Hämtad: 2026-04-14. 2026.
- [17] David Howells and Paul E. McKenney. *Circular Buffers — Linux Kernel Documentation*. <https://www.kernel.org/doc/html/v6.1/core-api/circular-buffers.html>. Hämtad: 2026-04-13.
- [18] Maged M. Michael och Michael L. Scott. "Nonblocking Algorithms and Preemption-Safe Locking on Multiprogrammed Shared Memory Multiprocessors". I: *Journal of Parallel and Distributed Computing* 51.1 (1998), s. 1–26. DOI: 10.1006/jpdc.1998.1446.
- [19] Abraham Silberschatz, Peter B. Galvin och Greg Gagne. *Operating System Concepts*. 10th. Hämtad: 2026-04-15. Wiley, 2018.
- [20] Michael Kerrisk. *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*. Hämtad: 2026-04-17. No Starch Press, 2010.
- [21] Irene Aldridge. "How Modern Markets Differ from Those Past: What Is High-Frequency Trading". I: *High-Frequency Trading: A Practical Guide to Algorithmic Strategies and Trading Systems*. Wiley, 2013. ISBN: 9780470579770.
- [22] Irene Aldridge. "Technological Innovations, Systems, and HFT: Speed and Security". I: *High-Frequency Trading: A Practical Guide to Algorithmic Strategies and Trading Systems*. Wiley, 2013. ISBN: 9780470579770.
- [23] Irene Aldridge. "Technological Innovations, Systems, and HFT: Software". I: *High-Frequency Trading: A Practical Guide to Algorithmic Strategies and Trading Systems*. Wiley, 2013. ISBN: 9780470579770.
- [24] Wang Hui, Li Dejian, Tang Xiaoke, Yang Lixin och Li Jingyun. "Research on Fast Inter-process Communication Algorithm". I: *2021 IEEE 2nd International Conference on Big Data, Artificial Intelligence and Internet of Things Engineering (ICBAIE)*. 2021, s. 156–162. DOI: 10.1109/ICBAIE52039.2021.9389942.
- [25] scrum.org. *Scrum, The scrum team, 3.2 What is sprint planning?* <https://www.scrum.org/learning-series/what-is-scrum/thescrum-events/what-is-sprint-planning>. Hämtad: 2026-03-24.
- [26] scrum.org. *Scrum, The scrum team, 3.4 What is sprint review?* <https://www.scrum.org/learning-series/what-is-scrum/the-scrum-events/what-is-a-sprintreview>. Hämtad: 2026-03-24.
- [27] OpenAI. *Codex CLI – Lightweight coding agent that runs in your terminal*. <https://github.com/openai/codex>. Hämtad: 2026-04-20. 2025.
- [28] Sida Peng, Eirini Kalliamvakou, Peter Cihon och Mert Demirer. "The Impact of AI on Developer Productivity: Evidence from GitHub Copilot". I: *arXiv preprint arXiv:2302.06590* (2023). DOI: 10.48550/arXiv.2302.06590.
- [29] fi.se. *Investigation into high frequency and algorithmic trading*. https://www.fi.se/contentassets/a9e47d166ba4466586bad3f216b46355/htf_eng.pdf. Hämtad: 2026-05-04.